



Basic Program Structure

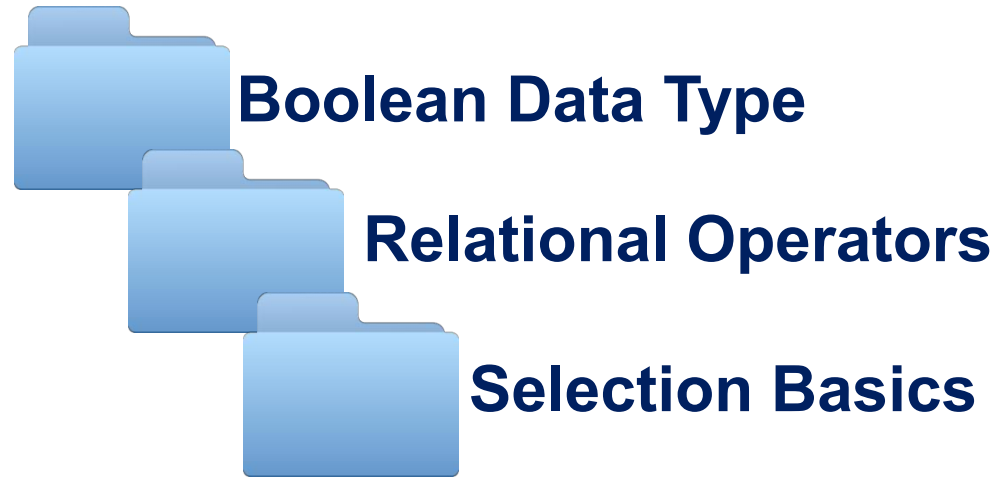
Boolean Data Type, Relational Operators and Selection Basics



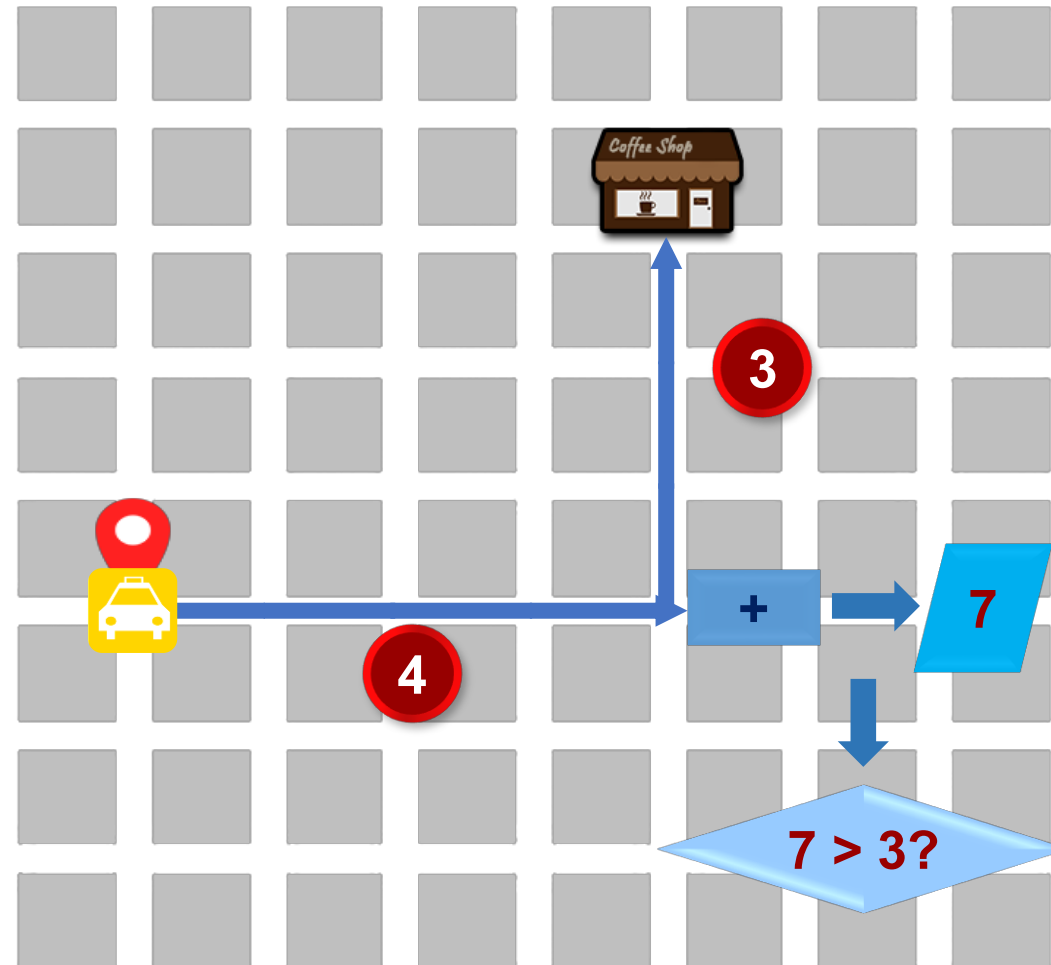
At the end of this lesson, you should be able to:

- Explain the usage of Boolean data type
- Use relational operators to form conditional expressions
- Discuss the basic concepts of program execution and flow control/ control flow
- Explain selection basics
- Describe the IF statement used for selection (branching)

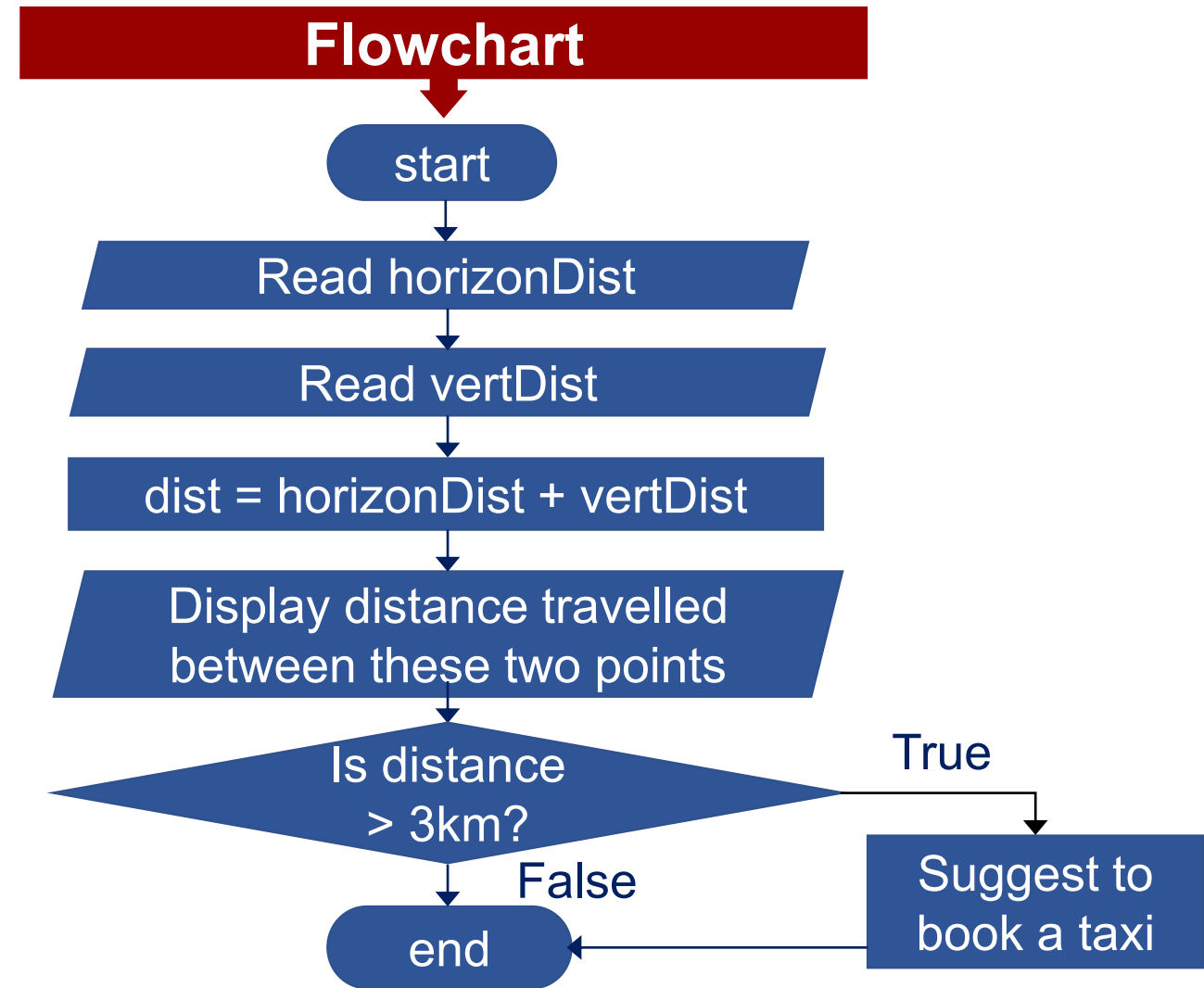
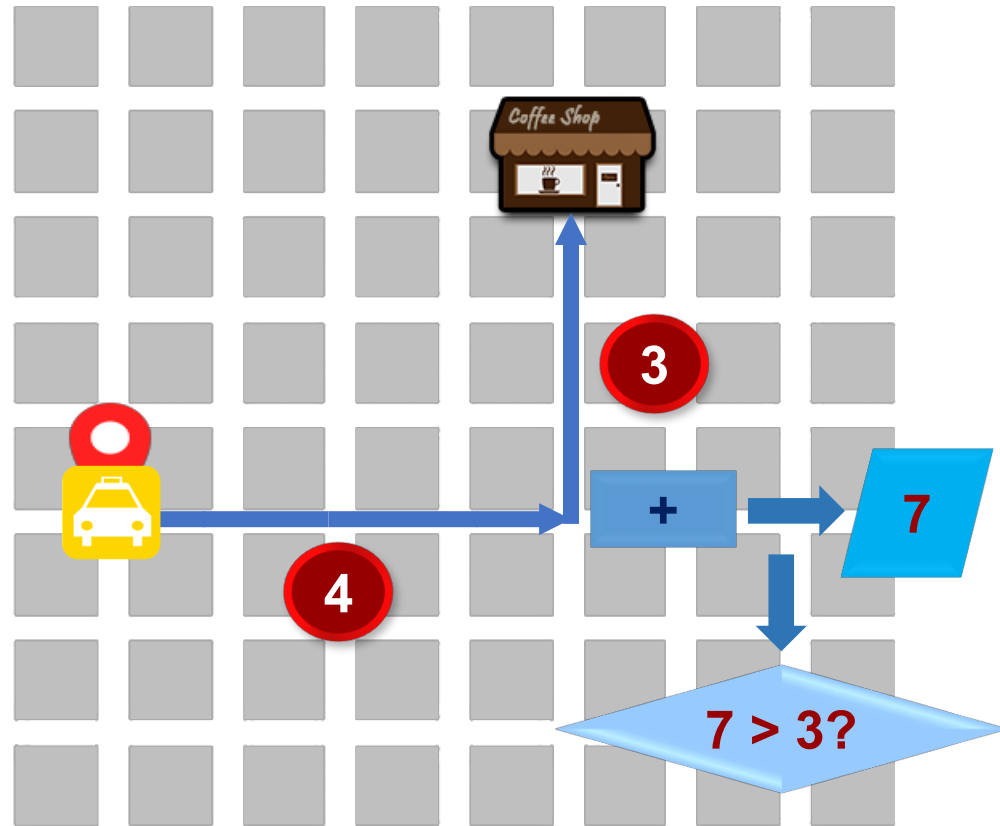
Topic Outline



Scenario 4: Decide if a Taxi is Needed

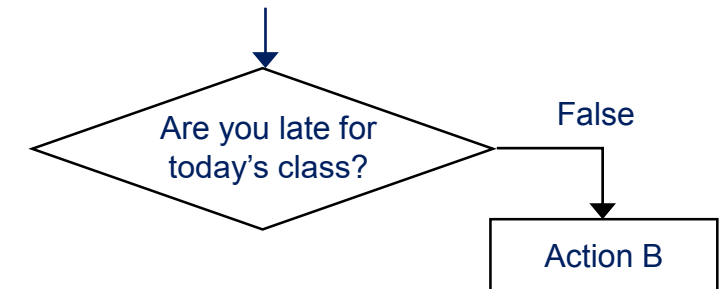
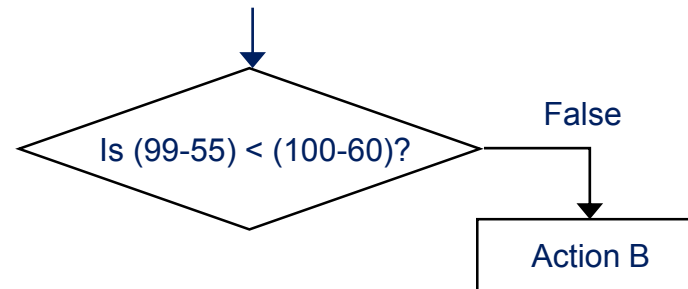
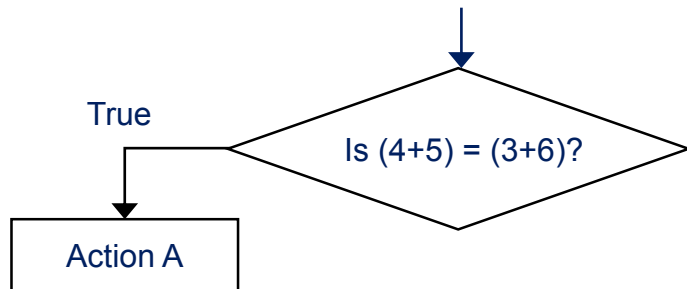
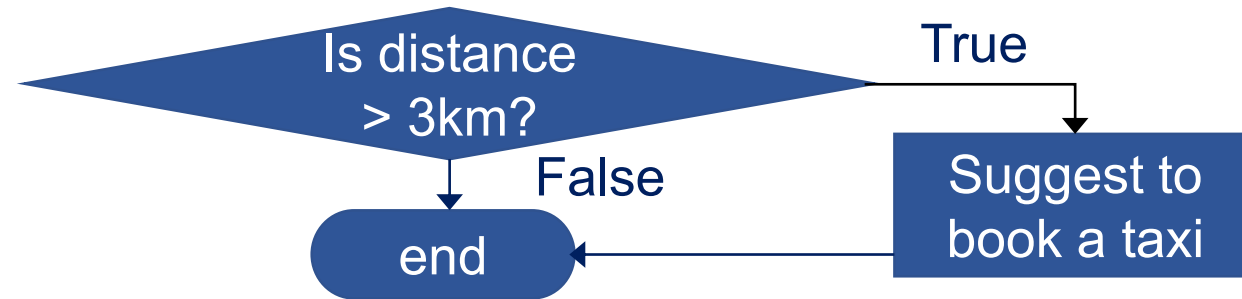


Scenario 4: Decide if a Taxi is Needed (Cont'd)

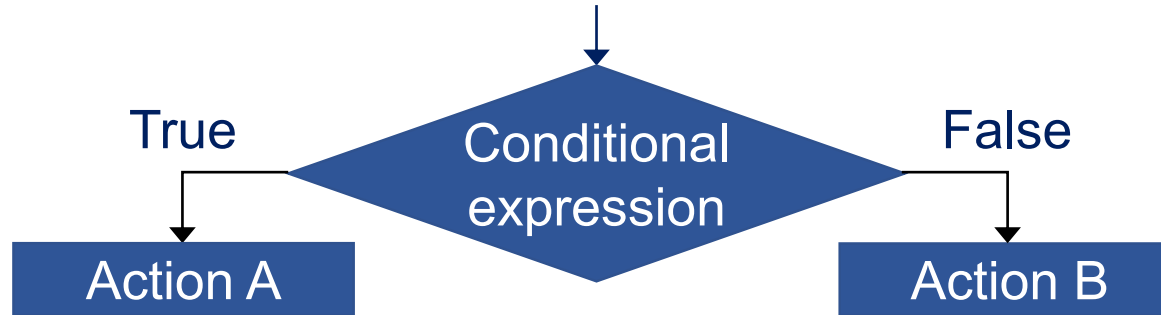


Conditional Statement

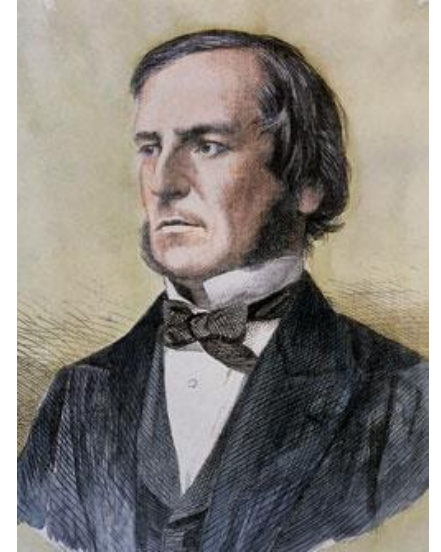
A **conditional statement** performs different actions depending on whether the condition evaluates to true or false.



Boolean Data Type



- In most computer programming languages, a **Boolean** data type is a data type with only two possible values, either **True** or **False**.
- Conditional expression, also called **Boolean expression**, may be composed of a combination of the **Boolean constants True or False**, **Boolean-typed variables**, **Boolean-valued operators**, and **Boolean-valued functions**.



George Boole is the author of The Laws of Thought that introduces Boolean algebra.



Can you think of any application of Boolean-valued operators?

Relational Operators

A **relational operator** compares two numbers (**float** or **int**) and returns a **Boolean** value of either **True** or **False**.

Relational Operator	Meaning	Example
<code>==</code>	equal to	<code>a == 1</code>
<code>!=</code>	not equal to	<code>b != 2</code>
<code><</code>	less than	<code>c < 3</code>
<code><=</code>	less than or equal to	<code>d <= 4</code>
<code>></code>	greater than	<code>f > 5.0</code>
<code>>=</code>	greater than or equal to	<code>f >= 6.0</code>

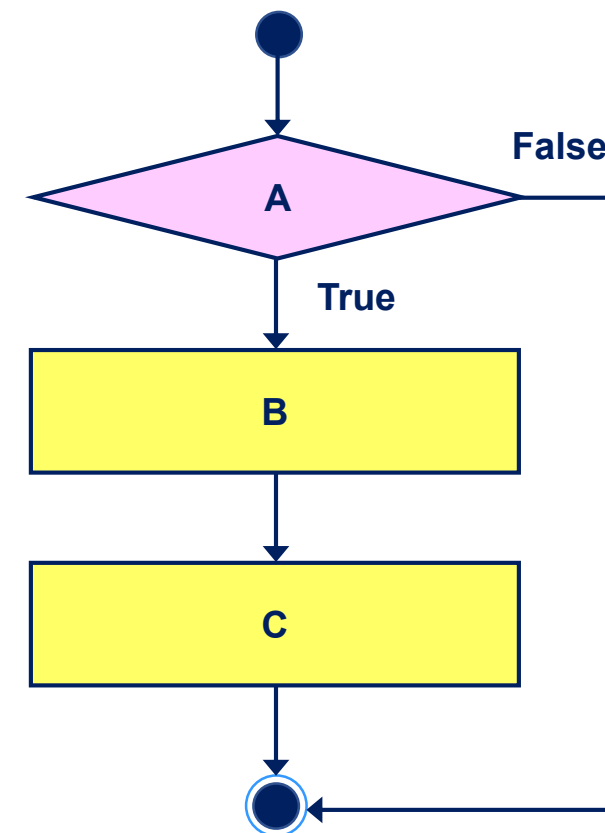
Program Execution and Flow Control/ Control Flow

- It controls which instruction should be executed next.
- By default, it is defined by the “sequence” concept, i.e., one after another.
- However, some structures can **alter** the flow, e.g., selection.
 - Selection (Branching) occurs when an “algorithm” makes a choice to do one of two or more things.

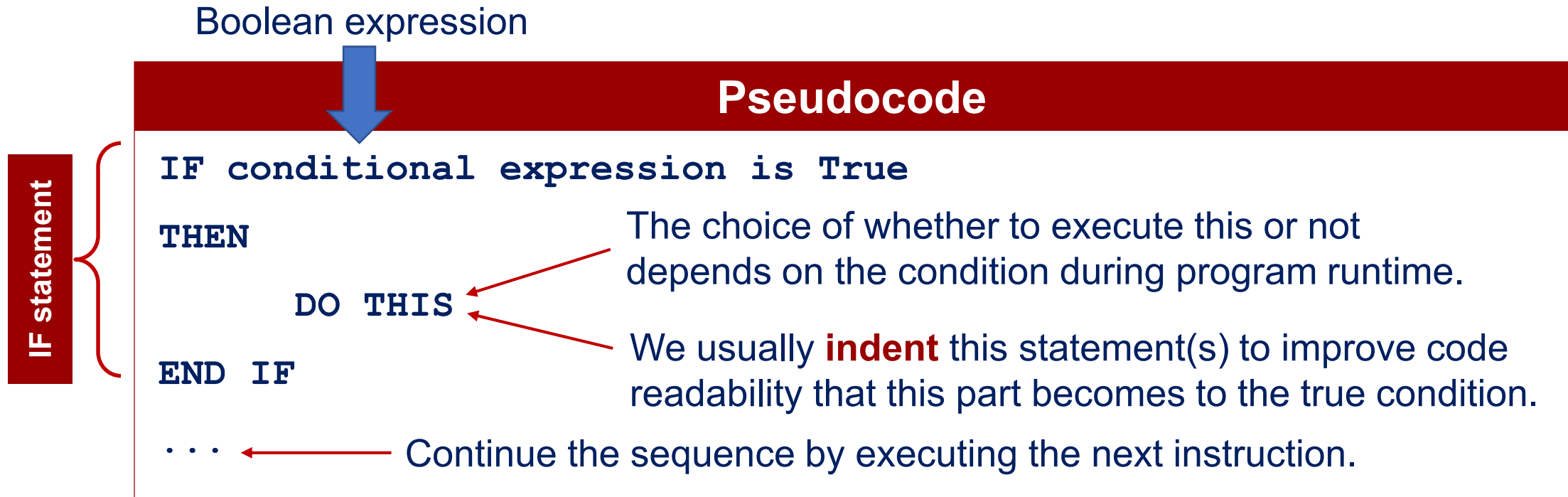


Message

- The flow control in a program is, in essence, **logic**.
- When writing or reading a program, ensure that you could **understand the flow**, i.e., what should be executed next for every step.



IF statement is the most common programming "statement" used for branching.



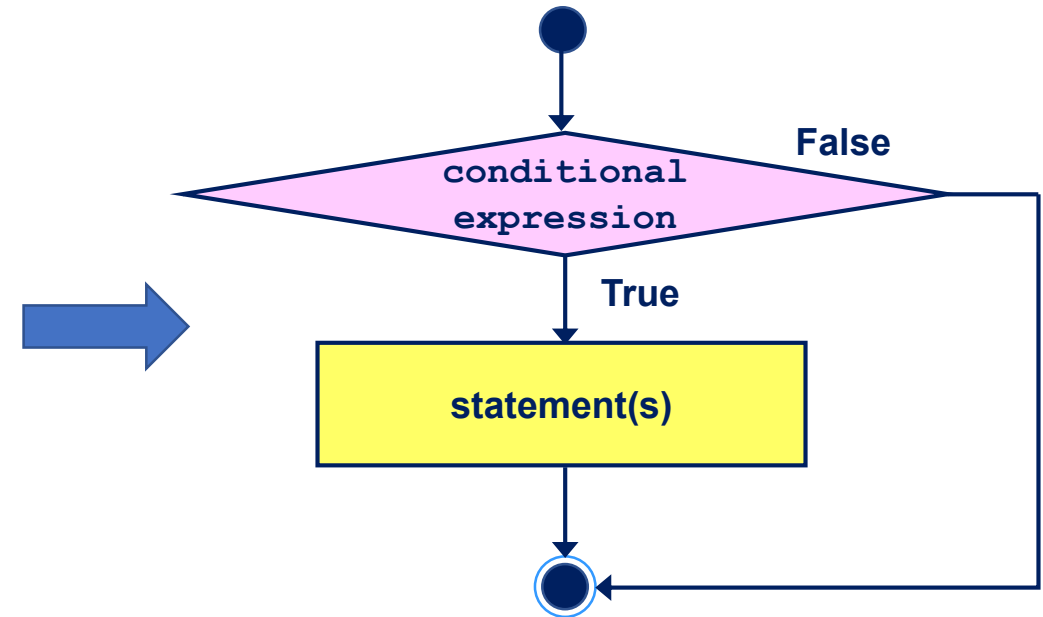
Note: Proper indentation is adopted in many programming languages.

IF Statement (Cont'd)

- Alternatively, when visualized as a flowchart:

Pseudocode

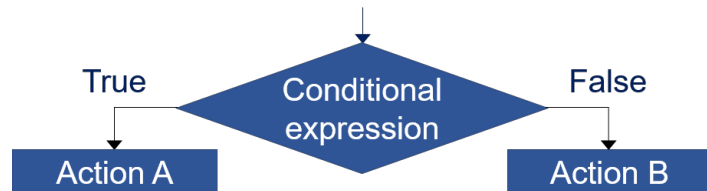
```
IF conditional expression is True
THEN
    DO THIS
END IF
...
```



- Hence, a program can make **decisions**!

Summary

Boolean Data Type



Relational Operators

==

!=

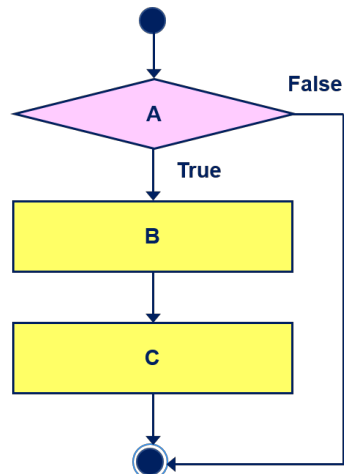
<

<=

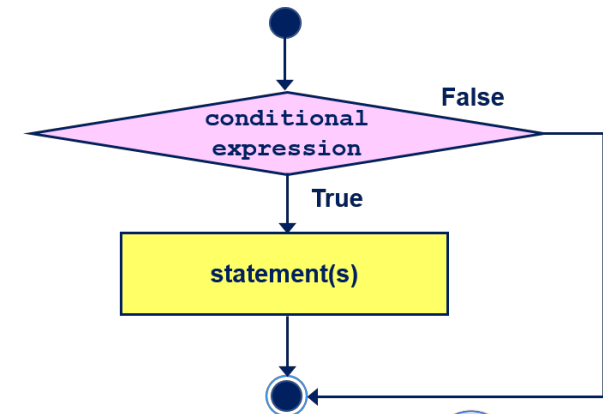
>

>=

Program Execution and Flow Control/ Control Flow



IF Statement



 *More on this later..*

References for the Images

No.	Slide No.	Image	Reference
1	8		Question problem [Online Image]. Retrieved April 24, 2018 from https://pixabay.com/en/question-problem-think-thinking-622164/ .
2	8		By Unknown - http://schools.keldysh.ru/sch444/museum/1_17-19.htm , Public Domain, retrieved April 18, 2018 from https://commons.wikimedia.org/w/index.php?curid=19667097 .
3	10		Warning [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/attention-warning-sign-danger-303861/ .
4	11, 13		By User:Bobarino - Made by following Information.png, CC BY-SA 3.0, retrieved April 28, 2014 from https://en.wikipedia.org/w/index.php?curid=9180601 .



Basic Program Structure

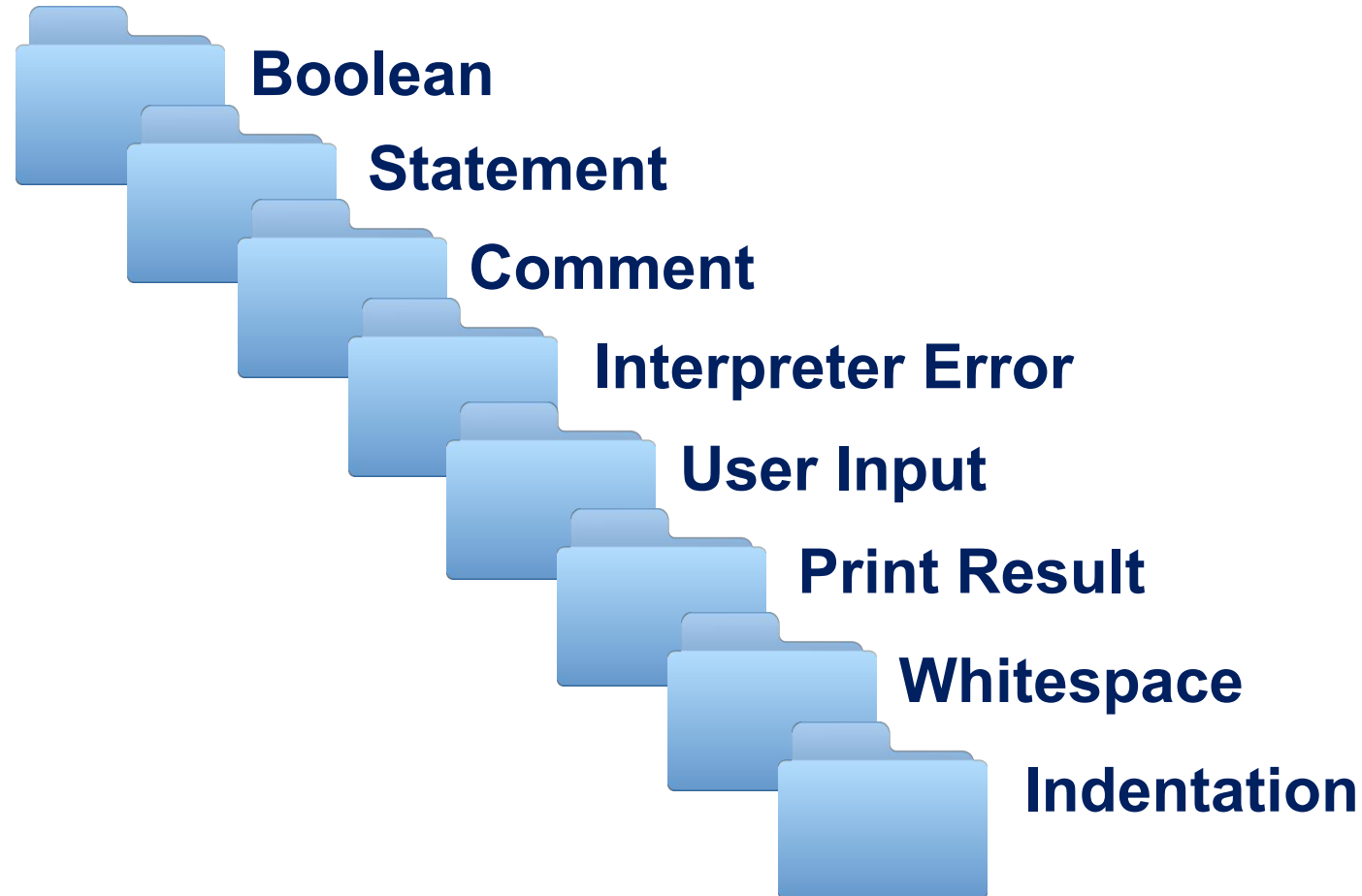
Boolean Data Type, Relational Operators, and Selection Basics - PYTHON



At the end of this lesson, you should be able to:

- Explain the following terminologies or syntax in Python:
 - Boolean data type
 - Relational operators
 - Selection basics
 - Comments using “#”
 - Built-in functions (e.g. input and print)
- Use the above terminologies or syntax in coding using Python

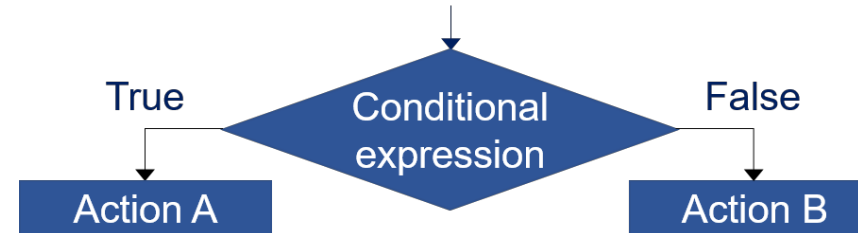
Topic Outline



Python Terminology #1: Boolean Data Type



- Designated as “**bool**”
- Either True or False



```
>>> a = True ← Use capital T for True.
```

```
>>> print(a)
```

```
True
```

```
>>> b = False ← Use capital F for False.
```

```
>>> c = true
```

```
Traceback (most recent call last):
```

```
  File "<pyshell#31>", line 1, in <module>
```

```
    c = true
```

```
NameError: name 'true' is not defined ←
```

This error occurs as
Python is case-sensitive.





What do you think is the output of the following Python code?

```
tom = 19  
print(tom > 18)
```



- A. true B. True C. false D. False

Quick Check: Answer



What do you think is the output of the following Python code?

```
tom = 19  
print(tom > 18)
```



A. true B. True C. false D. False

Answer

B. True

Python Terminology #2: Statement

```
# 1. prompt user for the radius
# 2. apply circumference and area formulae
# 3. print the results
```

```
import math
```

```
radiusString = input("Enter the radius of your circle:")
```

```
radiusFloat = float(radiusString)
```

```
circumference = 2 * math.pi * radiusFloat
```

```
area = math.pi * radiusFloat * radiusFloat
```

```
print()    # print a line break
```

```
print("The circumference of your circle is: ", circumference, \
      ", and the area is:", area)
```



← This line of code is one statement.

← This is another statement.

← This is one statement.

- Each line of code in a Python program is called a **statement**.
- Python **interprets** and **runs** the statements one by one.

Python Terminology #2: Statement (Cont'd)

```
# 1. prompt user for the radius
# 2. apply circumference and area formulae
# 3. print the results

import math
radiusString = input("Enter the radius of your circle:")
radiusFloat = float(radiusString)
circumference = 2 * math.pi * radiusFloat
area = math.pi * radiusFloat * radiusFloat

print()    # print a line break
print("The circumference of your circle is: ", circumference, "\n", and the area is:", area)
```



- The symbol “\” is used to **continue a statement with the next** line so that the two lines can be joined as one statement.
- It improves readability in the text editor.

Python is **sensitive to end of line** in text files, which marks the end of a statement.
In text editors, we press “Enter”.

Quick Try: Intelligent Guess



What do you think is the output of the following Python code?

```
print("The area") # print(" 18")
```



A. The area

B. The area 18

C. Syntax error

Quick Try: Answer



What do you think is the output of the following Python code?

```
print("The area") # print(" 18")
```



A. The area

B. The area 18

C. Syntax error

Answer

A. The area

Python Terminology #3: Comment

```
# 1. prompt user for the radius  
# 2. apply circumference and area formulae  
# 3. print the results
```

```
import math  
radiusString = input("Enter the radius of your circle:")  
radiusFloat = float(radiusString)  
circumference = 2 * math.pi * radiusFloat  
area = math.pi * radiusFloat * radiusFloat  
  
print() # print a line break  
print("The circumference of your circle is: ", circumference,\  
      ", and the area is:", area)
```

These are comment lines. 

- The pound sign “#” in Python indicates a **comment**.
- **Anything after “#”** is ignored during interpretation.

Comments provide information to improve code *readability*.

Python Terminology #3: Comment (Cont'd)



How can you add comments to the following Python code?

```
horizon_dist = (int)(input("Read horizonDist"))  
vertical_dist = (int)(input("Read vertDist"))  
travel_dist = horizon_dist + vertical_dist  
print("distance from A to B is", travel_dist)
```



```
#Calculate travel distance between two points  
#Date: 11/04/2018
```

```
horizon_dist = (int)(input("Read horizonDist"))  
vertical_dist = (int)(input("Read vertDist"))  
travel_dist = horizon_dist + vertical_dist  
print("distance from A to B is", travel_dist)
```



Python Terminology #4: Interpreter Error

The **interpreter** translates the Python code into machine language.

- The first stage of the process is to determine whether syntax of the code is valid or not.
- If the code is somehow invalid or malformed, Python cannot run the code and an **interpreter error** is received.

```
>>> print("there are no errors here")
there are no errors here
>>> print(but there is error here)
SyntaxError: invalid syntax
>>> print "more error here"
SyntaxError: invalid syntax
```



Syntax error: Double quotes are missing.

Syntax error: Brackets are missing.

 **Syntax error:** Python cannot translate the code.

Quick Try: Intelligent Guess



What do you think is the output of the following Python code if the user inputs “25”?

```
date = input("date of today?")  
thedayaftertomorrow = date + 2  
print(thedayaftertomorrow)
```



- A. 27 B. Error C. 25

Quick Try: Answer



What do you think is the output of the following Python code if the user inputs “25”?

```
date = input("date of today?")  
thedayaftertomorrow = date + 2  
print(thedayaftertomorrow)
```



- A. 27 B. Error C. 25

Answer

B. Error

Python Terminology #5: User Input

An **input** is a **built-in function**, to get an input provided by Python, that:

- **prints** the message string on the screen and **waits until the user types** anything and **presses “Enter”**.
- returns a **string** no matter what is given, even a number.

```
# 1. prompt user for the radius
# 2. apply circumference and area formulae
# 3. print the results

import math

radiusString = input("Enter the radius of your circle:")
radiusFloat = float(radiusString)
circumference = 2 * math.pi * radiusFloat
area = math.pi * radiusFloat * radiusFloat

print()    # print a line break
print("The circumference of your circle is: ", circumference,
      ", and the area is:", area)
```





What do you think is the output of the following modified Python code if the user inputs “25”?

```
date = input("date of today?")  
thedayaftertomorrow = int(date) + 2  
print(thedayaftertomorrow)
```



- A. 27 B. Error C. 25

Quick Check: Answer



What do you think is the output of the following modified Python code if the user inputs “25”?

```
date = input("date of today?")  
thedayaftertomorrow = int(date) + 2  
print(thedayaftertomorrow)
```



A. 27

B. Error

C. 25

Answer

A. 27

Python Terminology #6: Print Result

Print is another **built-in function**, provided by Python, that displays related message and data on the shell screen.

- It uses a **comma** to separate the elements.

```
# 1. prompt user for the radius
# 2. apply circumference and area formulae
# 3. print the results

import math
radiusString = input("Enter the radius of your circle:")
radiusFloat = float(radiusString)
circumference = 2 * math.pi * radiusFloat
area = math.pi * radiusFloat * radiusFloat
print("radius is: ")
print(radiusFloat)
print() # print a line break
print("The circumference of your circle is: " + circumference + "\n", and the area is:" + area)
```



The function **“print()”** makes an empty line.



*What happens if you use several **“print()”** functions?*



Pseudocode

IF condition is True

THEN

DO THIS

END IF

...

We usually **indent** this/ these statement(s) to improve code readability that this part becomes code body of the true condition.



Note: Proper indentation is adopted in many programming languages. However, Python forces you to use proper indentation.

Syntax

if < a Boolean expression> : ← A colon **must** be used to mark the start of a block.
one or more indented statements ← An indentation **must** be used for the entire true block.



Examples

```
if a > b:  
    print("a > b")
```

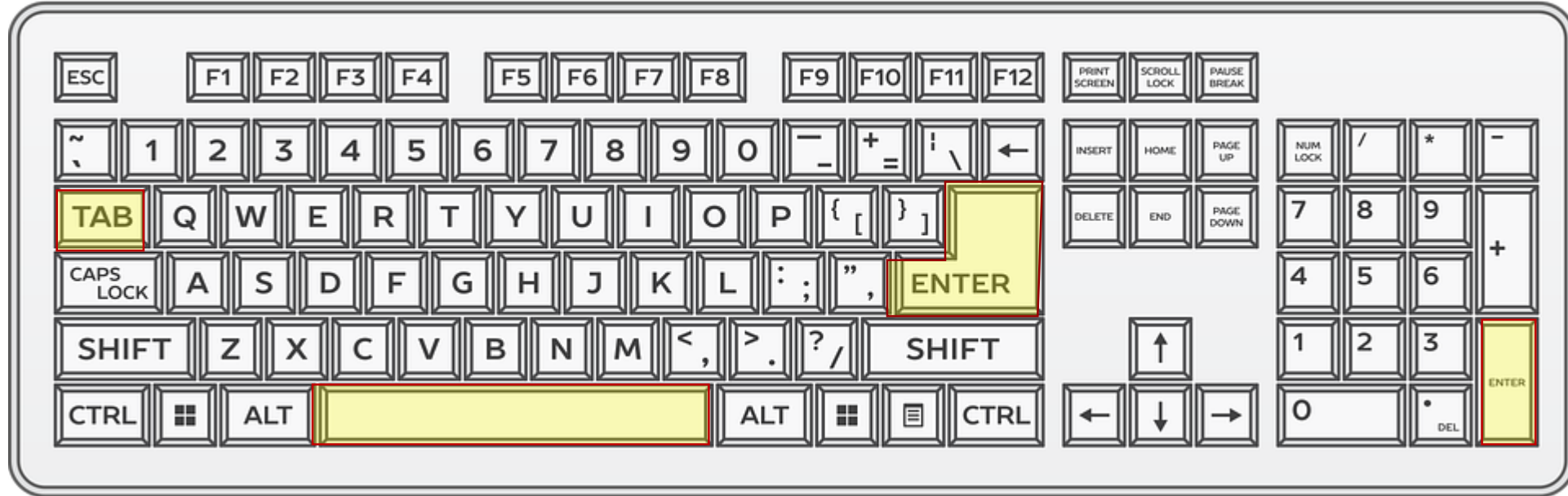


```
bodyTemp = (float)(input("Read body  
temperature"))  
if bodyTemp >= 37.5:  
    print("Fever")  
    print("Time to see a doctor!")  
print("End of program")
```



Python Terminology #7: Whitespace

Python counts the “Tab”, “Spacebar” and “Enter” as **whitespaces**:



- **Purpose of whitespace:** **separates** words in a statement
- For the most part, you can place whitespaces anywhere in your program to make the code more **readable**. E.g., use `a = a + 1 + c` instead of `a=a+1+c`

Quick Try: Intelligent Guess



Is the following Python code correct?

```
print(" H1")  
    print()  
print("H2")
```



Quick Try: Answer

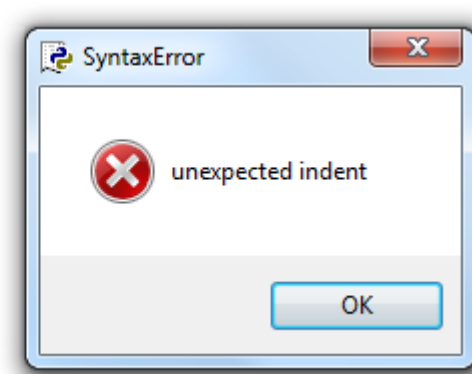


Is the following Python code correct?

```
print(" H1")  
    print()  
print("H2")
```



```
print(" H1")  
print()  
print("H2")
```



Answer

No

Python Terminology #8: Indentation

An **indentation** is a leading whitespace at the start of a statement.

Pseudocode

```
IF body temperature > 37.5
THEN
    print("Fever!")
    print("Time to see a doctor!")
END IF
...
```



Note: Use the same number of spaces for indentation **consistently**.

In Python, this group of indented statement(s) is called a **suite** or a **block**.

A **compound statement** is a set of statements being used as a group.

Purpose of Indentation:

- For making the code more **readable**
- For **grouping**, particularly for **control flow** such as **branching** and **looping**

Python Terminology #8: Indentation for Grouping

```
N = int(input("Input N: "))
if N <= 1:
    NFact = 1
else:
    NFact = 2
    for i in range(3, N + 1):
        NFact = NFact * i
print(NFact)
```



One level of indentation

Two levels of indentation

- The power of an IF statement is that the program can **selectively run** the block based on the runtime condition.
- Sometimes the program runs it, sometimes the program skips it!



What is the output of the following Python code?

```
a = 3
if a > 5:
    print(a)
    print(a + 3)
print(a + 5)
```

- A. 3
- B. 6
- C. 8
- D. Syntax error
- E. A & B & C
- F. B & C

Quick Check: Answer



What is the output of the following Python code?

```
a = 3
if a > 5:
    print(a)
    print(a + 3)
print(a + 5)
```

- A. 3
- B. 6
- C. 8
- D. Syntax error
- E. A & B & C
- F. B & C

Answer

C. 8

Recap

- A **colon** marks the start of a block.
- In Python, an **indentation** is not just for readability but also for defining the scope of a block.
- The number of **whitespace** for indentation is flexible but should be consistent for the same level in the same program.

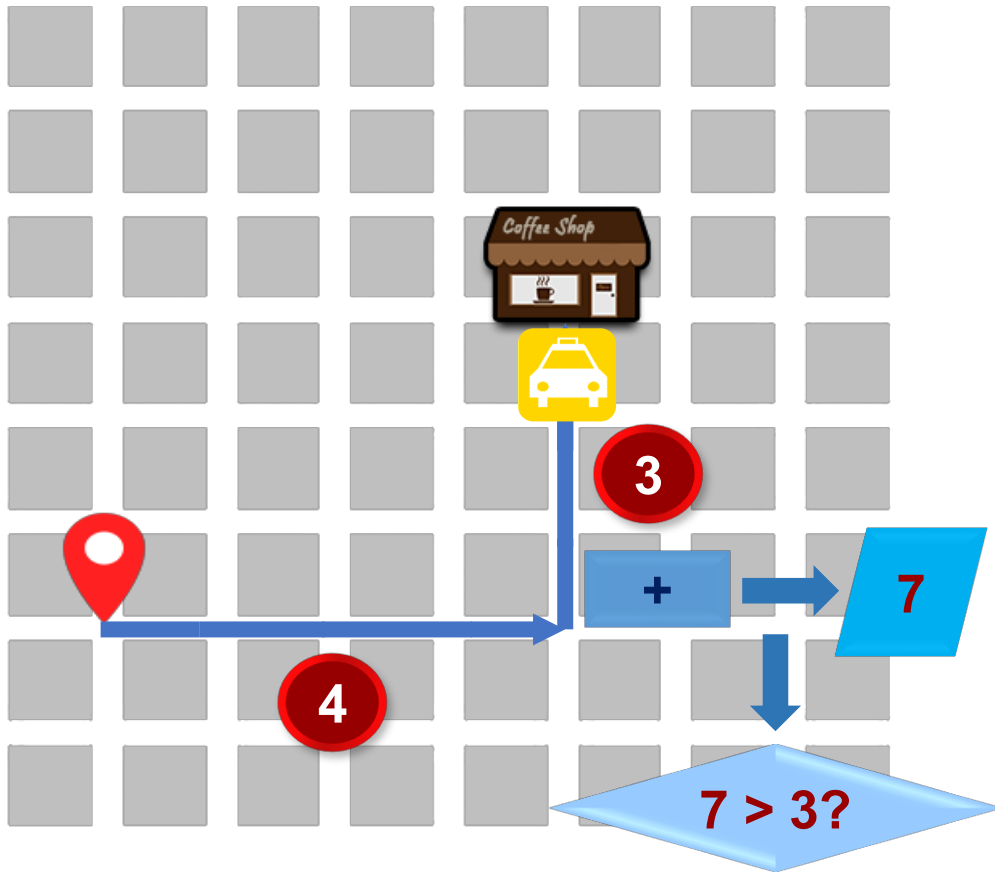
```
if a > b:  
    print("a > b")  
    print("still this block")
```



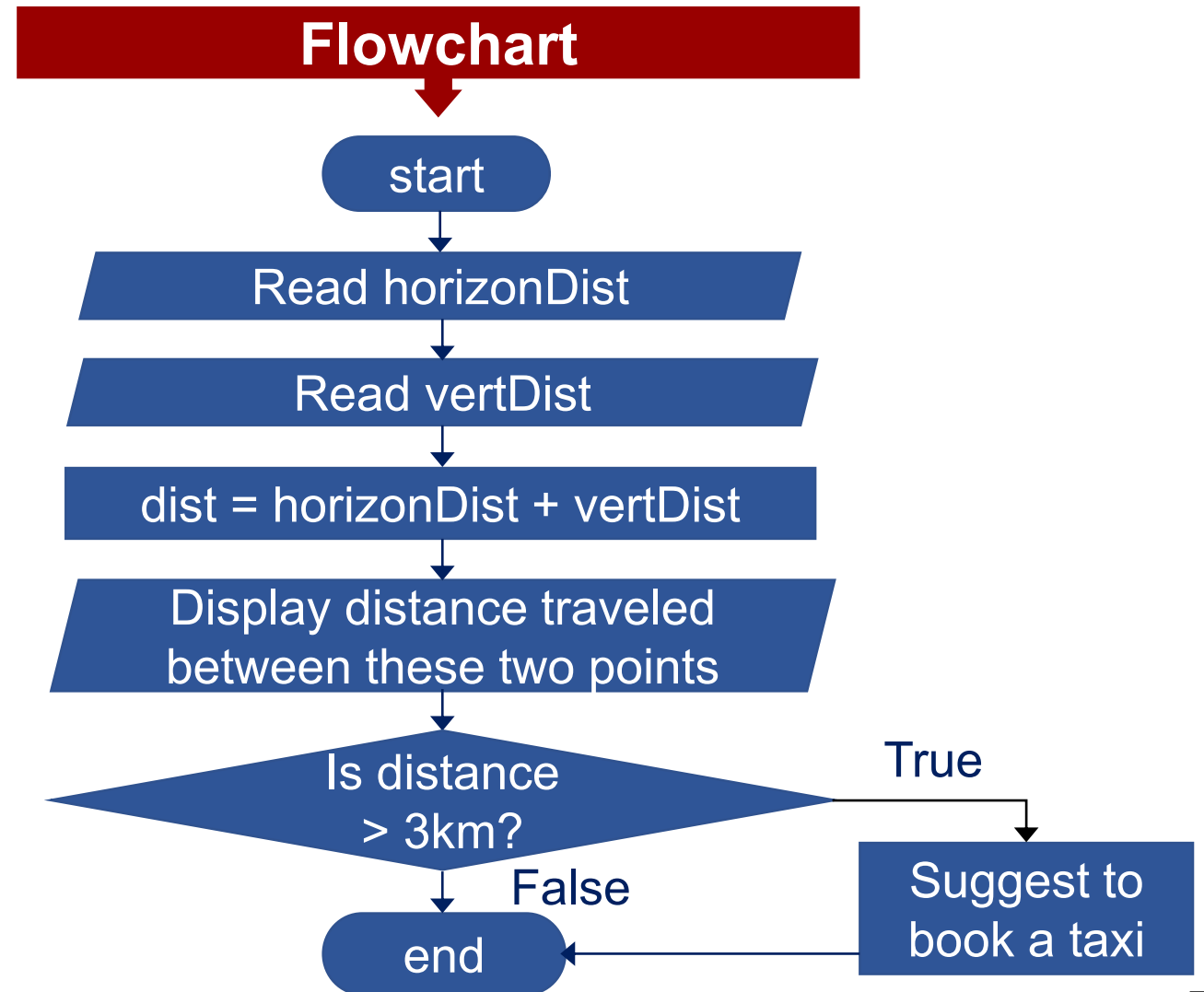
Scenario 4: Decide if a Taxi is Needed



Recall



Flowchart



Scenario 4: Decide if a Taxi is Needed - Python Code

```
#Decide if a Taxi is Needed
#Date: 11/04/2018
horizon_dist = (int)(input("Read horizonDist"))
vertical_dist = (int)(input("Read vertDist"))
travel_dist = horizon_dist + vertical_dist
print("distance from A to B is ", travel_dist, "m")
if travel_dist > 3000:
    print("You are suggested to book a taxi")
print("Thank you for using this application")
```



Output when True

Read horizonDist in meters 4000
Read vertDist in meters 3000
distance from A to B is 7000 m
You are suggested to book a taxi
Thank you for using this application

Scenario 4: Decide if a Taxi is Needed - Python Code

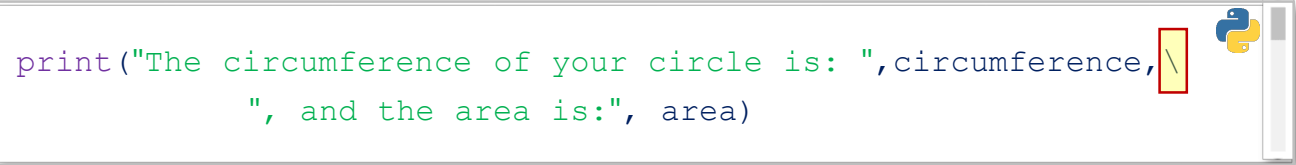
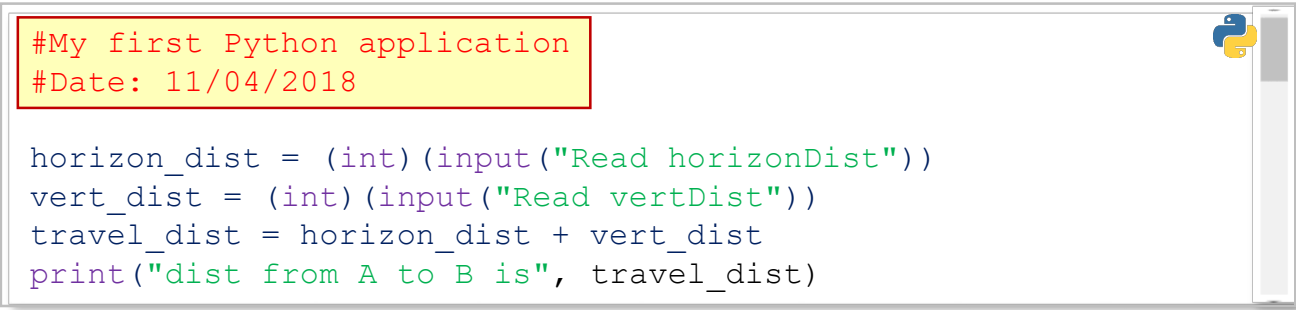
```
#Decide if a Taxi is Needed
#Date: 11/04/2018
horizon_dist = (int)(input("Read horizonDist"))
vertical_dist = (int)(input("Read vertDist"))
travel_dist = horizon_dist + vertical_dist
print("distance from A to B is ", travel_dist, "m")
if travel_dist > 3000:
    print("You are suggested to book a taxi")
print("Thank you for using this application")
```



Output when False

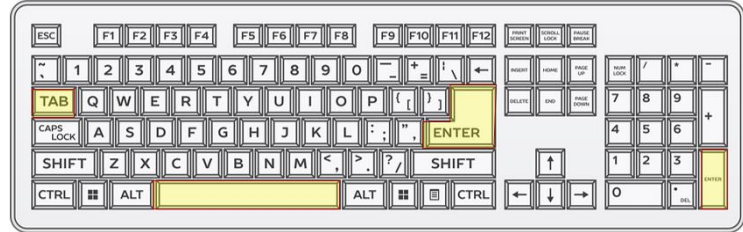
Read horizonDist in meters 4000
Read vertDist in meters 3000
distance from A to B is 7000 m
Thank you for using this application

Summary

Terminology	Description	Illustrative Example
Boolean	Either True or False	
Statement	A line of code; uses “\” to continue a statement with the next line so that the two lines can be joined as one statement.	
Comment	Indicated by a “#” and ignored during interpretation	

Summary

Terminology	Description	Illustrative Example
Interpreter error	Occurs when the code is invalid	<pre>>>> print(but there is error here) SyntaxError: invalid syntax >>> print "more error here" SyntaxError: invalid syntax</pre>
User input	Prints the message string on the screen and waits until the user types anything as input and presses “Enter” <i>Note: In Python 2, the function call is <code>raw_input</code>, instead of <code>input</code>.</i>	<pre>import math radiusString = input("Enter the radius of your circle:") radiusFloat = float(radiusString) circumference = 2 * math.pi * radiusFloat area = math.pi * radiusFloat * radiusFloat</pre>
Print result	Displays related message and data on the shell screen	<pre>print() # print a line break print("The circumference of your circle is: ", circumference, \ ", and the area is:", area)</pre>

Terminology	Description	Illustrative Example
Whitespace	A character that separates words in a statement (Tab, Spacebar and Enter)	
Indentation	A leading whitespace at the start of a statement for grouping and making the code more readable	<pre> N = int(input("Input N: ")) if N <= 1: NFact = 1 else: NFact = 2 for i in range(3, N + 1): NFact = NFact * i print(NFact) </pre>

References for Images

No.	Slide No.	Image	Reference
1	5, 21, 31		Search [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/database-search-database-search-icon-2797375/ .
2	All pages with Python codes		Python Logo [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/language-logo-python-2024210/ .
3	13, 20		Question problem [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/question-problem-think-thinking-622164/ .
4	21, 26		By User:Bobarino - Made by following Information.png, CC BY-SA 3.0, retrieved April 18, 2018 from https://en.wikipedia.org/w/index.php?curid=9180601 .
5	22		Survey icon [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/survey-icon-survey-icon-2316468/ .

References for Images

No.	Slide No.	Image	Reference
6	23, 36		Keyboard [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/keyboard-the-keys-on-the-keyboard-2170063/ .



Basic Program Structure

More on Selection (Branching)



At the end of this lesson, you should be able to:

- Apply the following selection (branching) structures:
 - IF-ELSE statement
 - Nested IF statement
- Use Logical/ Boolean Operators to chain expressions

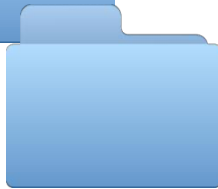
Topic Outline



IF-ELSE Statement

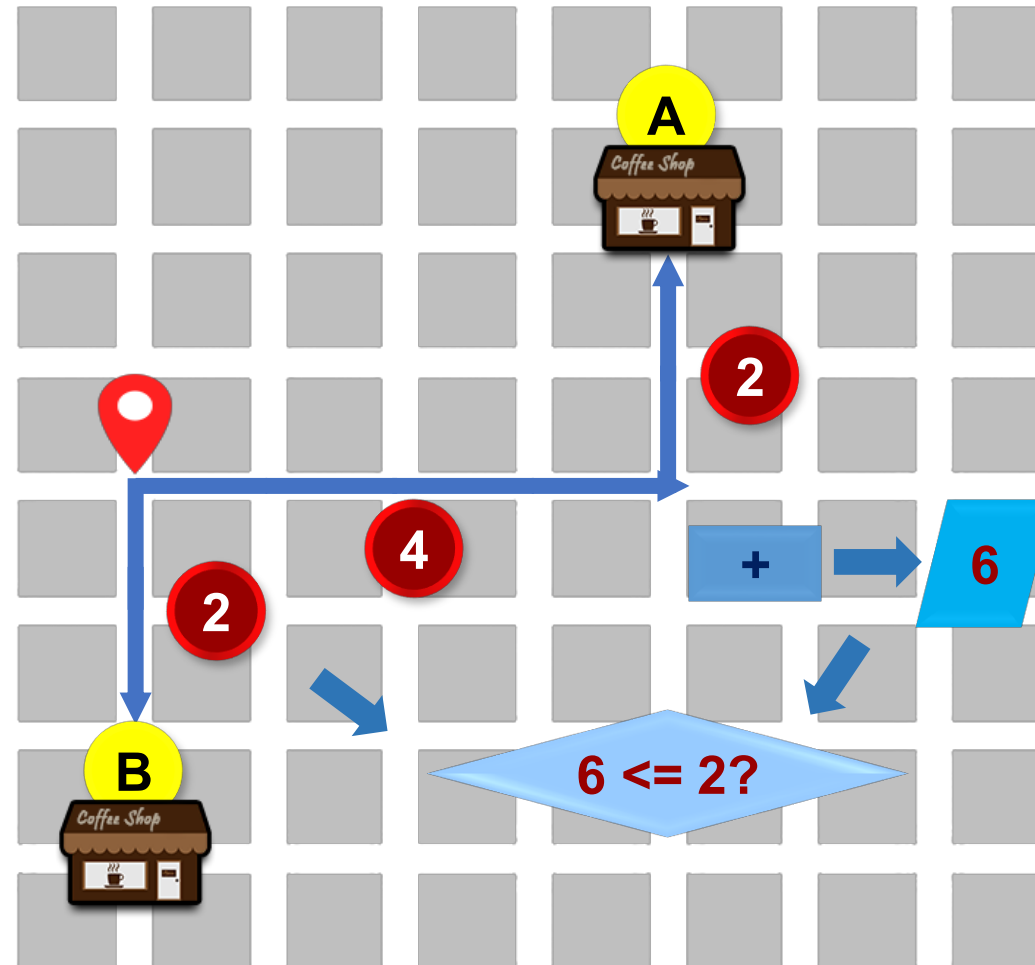


Boolean Operators

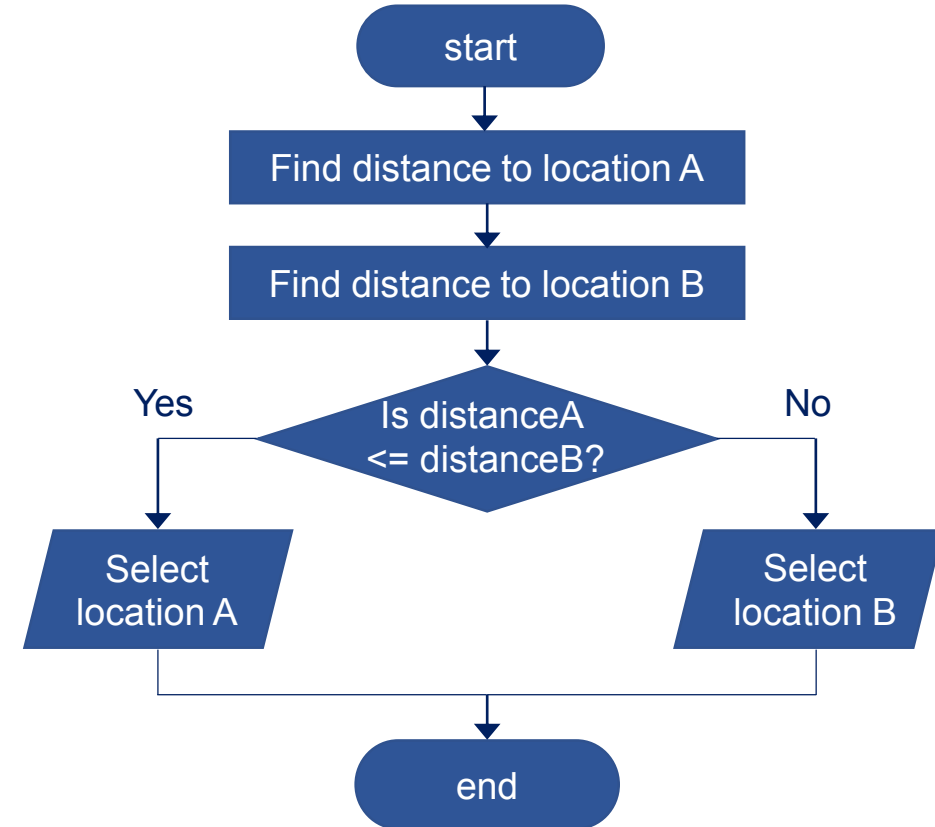
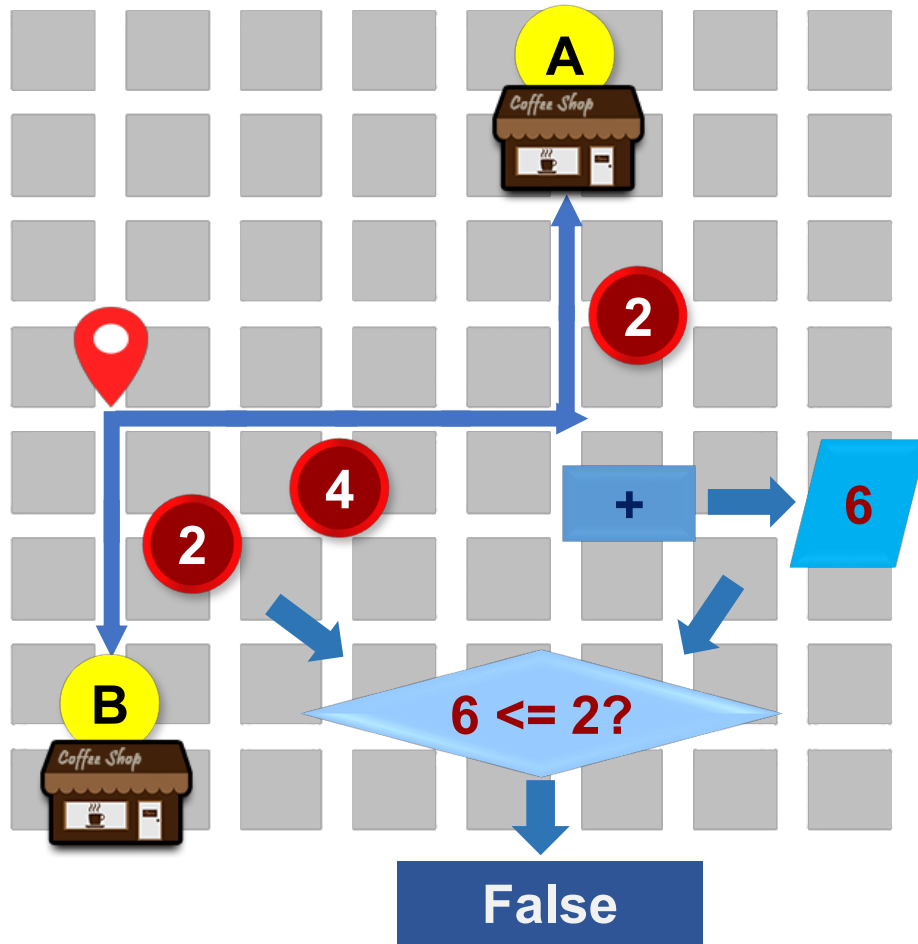


Nested IF Statement

Scenario 5: Find the Nearer Coffee Shop



Scenario 5: Find the Nearer Coffee Shop (Cont'd)



IF-ELSE Statement

A single
IF-ELSE
statement in
the top level

Pseudocode

IF condition is True

THEN

 **SUITE1** ← Execute the True part if the condition is True.

ELSE

Note the same amount of indentation used.

 **SUITE2** ← Execute the False part if the condition is False.

END IF

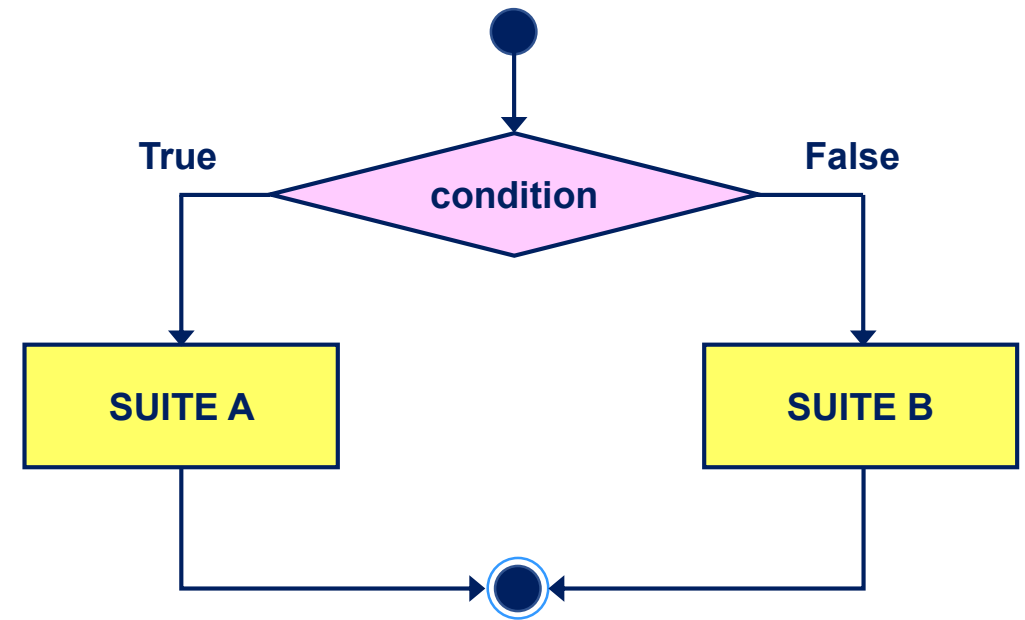
... ← Continue the sequence by executing the next instruction.

IF-ELSE Statement (Cont'd)

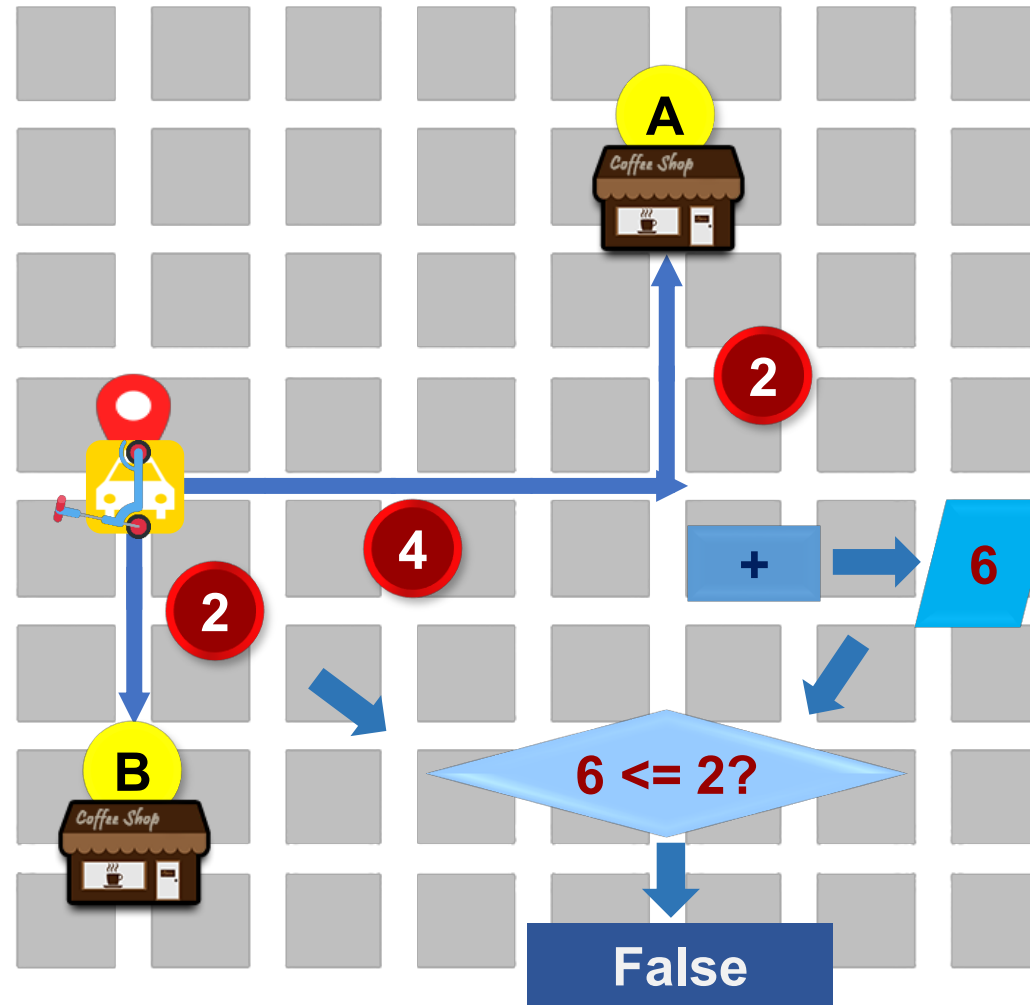
Alternatively, when visualized as a flowchart:

Pseudocode

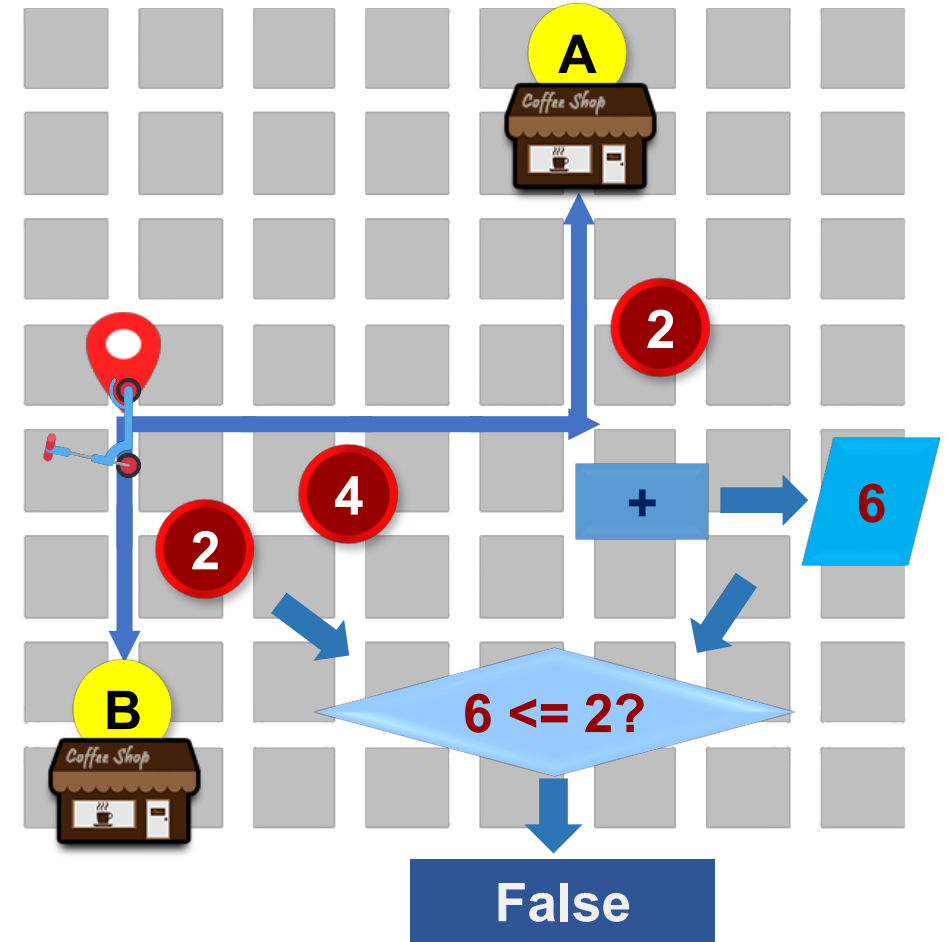
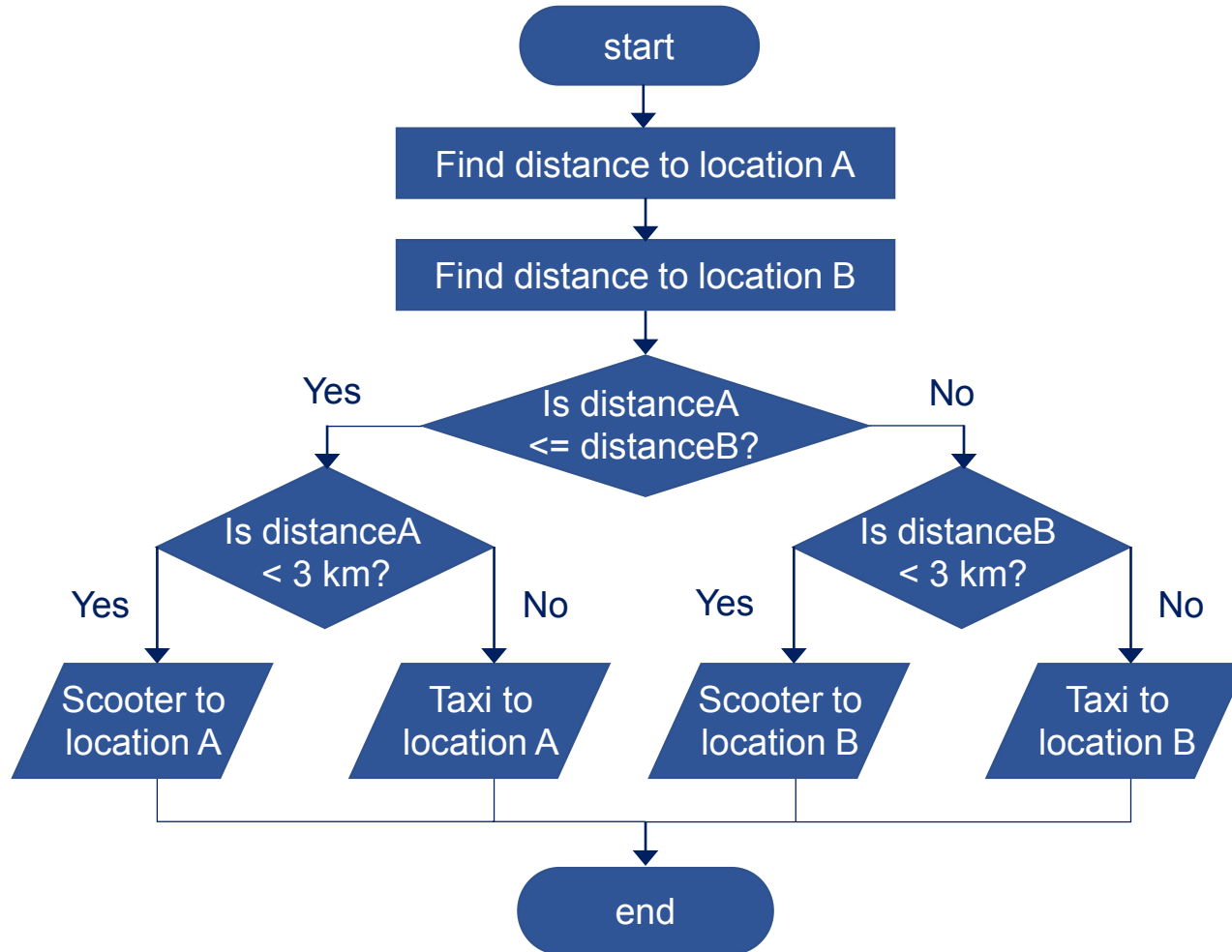
```
IF condition is True
THEN
    SUITE A
ELSE
    SUITE B
END IF
...
```



Scenario 6: Decide Transportation to the Nearer Coffee Shop



Scenario 6: Decide Transportation to the Nearer Coffee Shop





Pseudocode

IF condition1 is True THEN

DO THIS SUITE ← For each IF statement, there is an associated block for its True part.

END IF

...

Logical/ Boolean Operators

Logical operators **connect** Boolean values and expressions and **return** a Boolean value as a result.

A	B	not A	A and B	A or B
False	False	True	False	False
False	True	True	False	True
True	False	False	False	True
True	True	False	True	True

Operator	Example	Meaning
not	not num < 0	Flip T/ F
and	(num1 > num2) and (num2 > num3)	Return True only if both are True
or	(num1 > num2) or (num2 > num3)	Return True if either one is True



What is the value of the given expression?

(**not** (10>=20) and **not** (10<30))

Quick Check: Answer



What is the value of the given expression?

(**not** (10>=20) **and** **not** (10<30))

Answer

False

Nested IF Statement

Inside the True Block

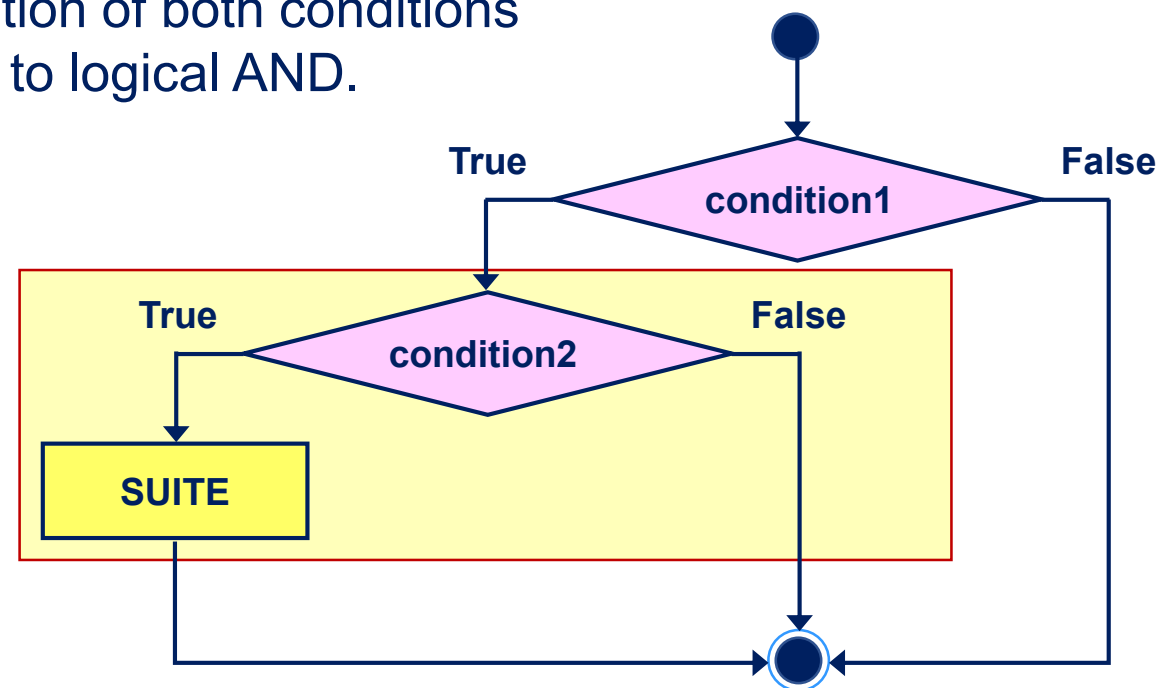


Can there be another IF statement(s) inside the **True Block**?

Pseudocode

```
IF condition1 is True THEN
    IF condition2 is True THEN
        DO THIS SUITE
    END IF
END IF
...
```

Combination of both conditions is similar to logical AND.



Nested IF Statement (Cont'd)

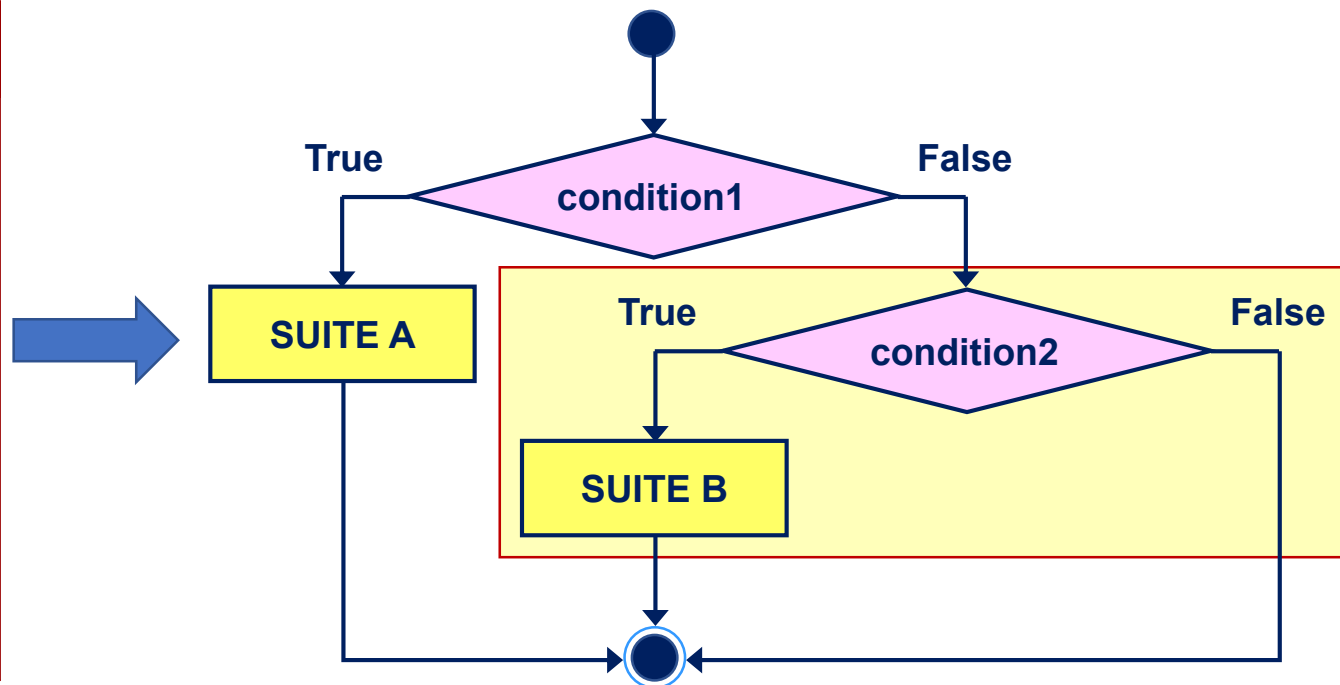
Inside the False Block



Can there be another IF statement(s) inside the **False Block**?

Pseudocode

```
IF condition1 is True THEN
    DO SUITE A
ELSE IF condition2 is True THEN
    DO SUITE B
END IF
END IF
...
```



Nested IF Statement (Cont'd)

Inside the True Block of the True Block



Can there be another IF statement(s) inside the **True Block of a True Block**?

Pseudocode

```
IF condition1 is True THEN
  IF condition2 is True THEN
    IF condition3 is True THEN
      DO THIS SUITE
    END IF
  END IF
END IF
...
```

We do this only if condition1, condition2, and condition3 are true, similar to logical AND.



Try creating its equivalent flowchart.
Use the previous flowchart as a reference.

Nested IF Statement (Cont'd)

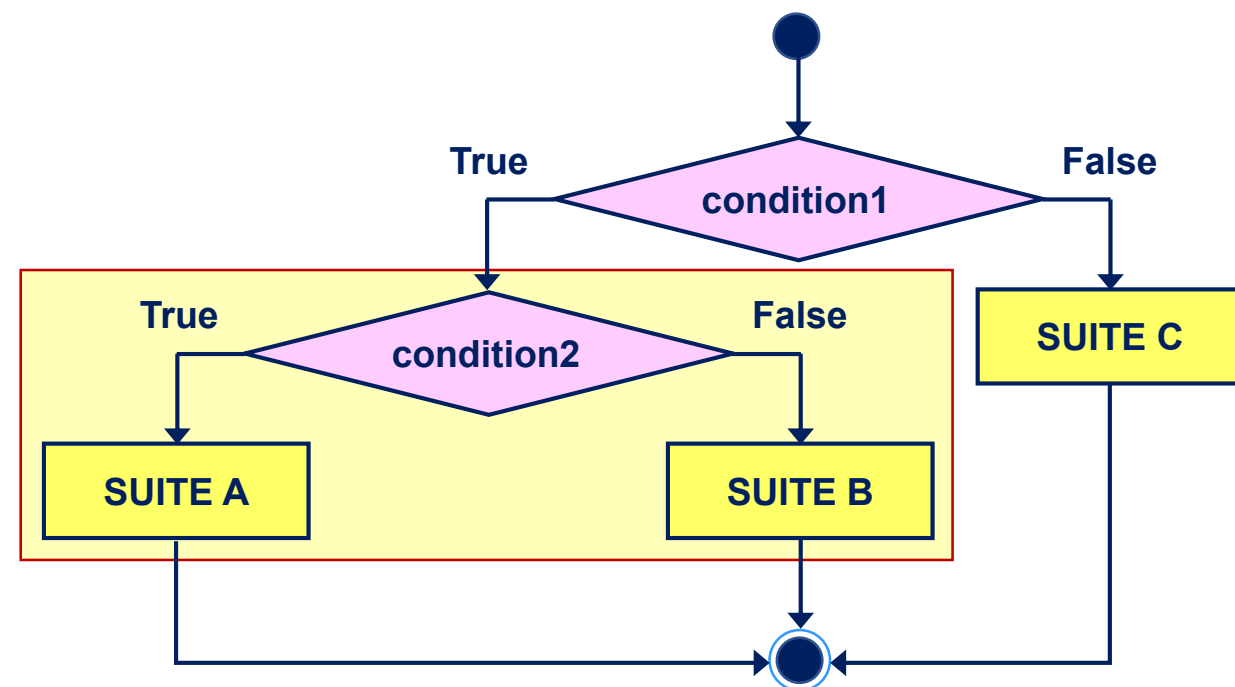
IF-ELSE Statement(s) Inside the True Block



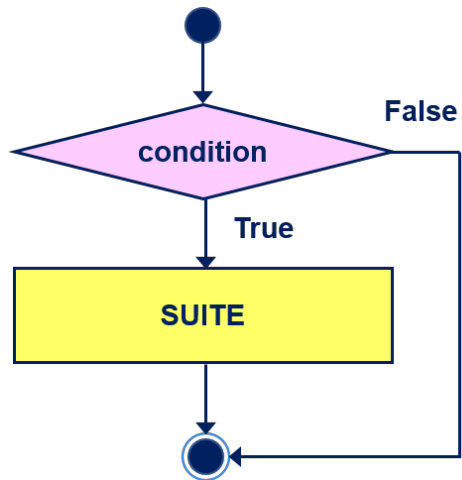
Can there be another IF-ELSE statement(s) inside the **True Block**?

Pseudocode

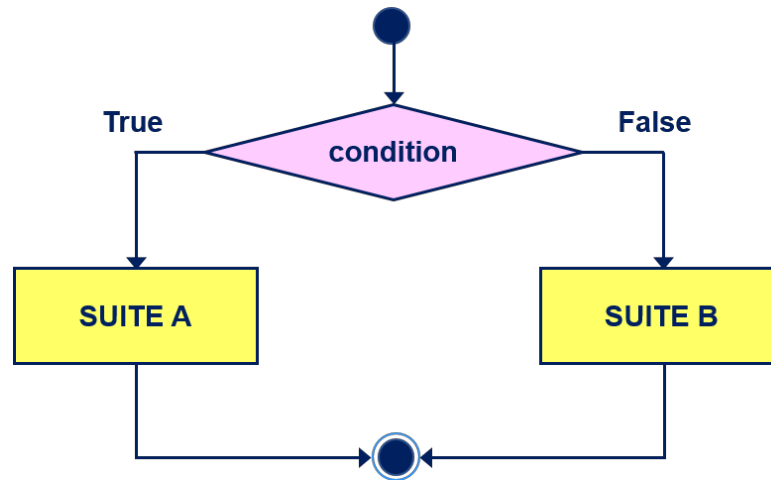
```
IF condition1 is True THEN
    IF condition2 is True THEN
        DO SUITE A
    ELSE
        DO SUITE B
    END IF
ELSE
    DO SUITE C
END IF
...
```



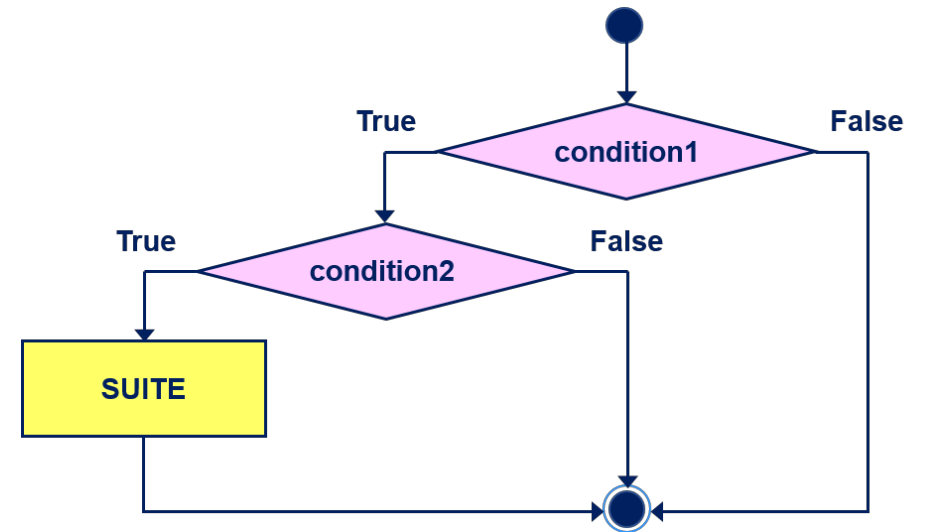
IF Statement



IF-ELSE Statement



Nested IF Statement



References for Images

No.	Slide No.	Image	Reference
1	11		Search [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/database-search-database-search-icon-2797375/ .
2	16, 17, 18, 19		Question problem [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/question-problem-think-thinking-622164/ .
3	9, 10		By Twitter, CC BY 4.0, retrieved April 18, 2018 from https://commons.wikimedia.org/w/index.php?curid=59970363 .
4	17		Survey icon [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/survey-icon-survey-icon-2316468/ .



Basic Program Structure

More on Selection (Branching) - PYTHON



At the end of this lesson, you should be able to:

- Apply the following selection (branching) structures in Python:
 - IF-ELSE statement
 - IF-ELIF-ELSE statement
 - Nested IF statement
- Explain all the basic arithmetic operators in Python
- Use the basic arithmetic operators to solve problems

Topic Outline



IF-ELSE Statement



Arithmetic Operators in Python



IF-ELIF-ELSE Statement

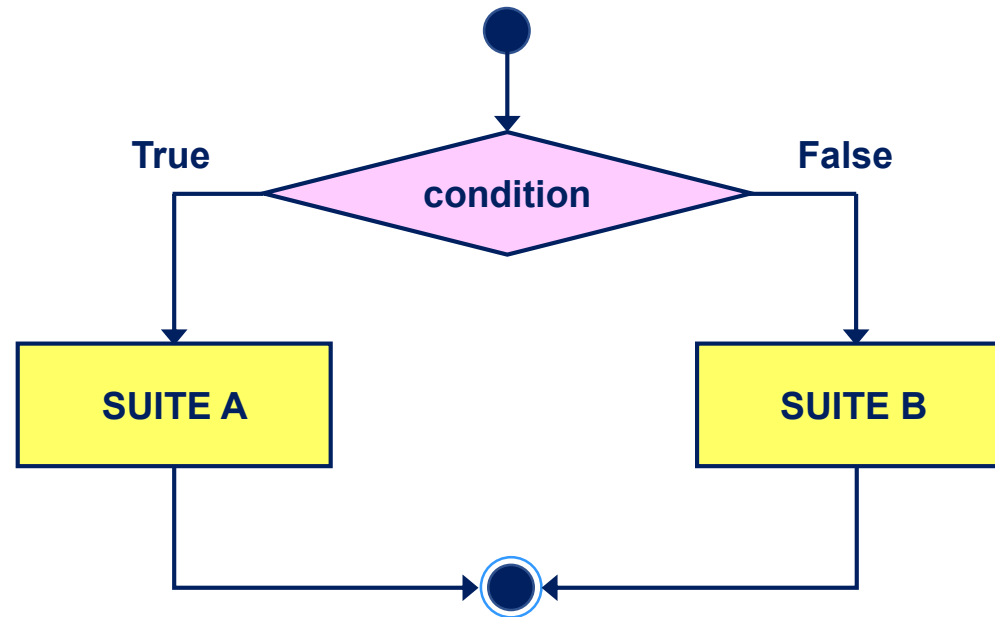


Nested IF Statement

IF-ELSE Statement (recall)

Pseudocode

```
IF condition is True
THEN
    SUITE A
ELSE
    SUITE B
END IF
...
```



IF-ELSE Statement - General Python Syntax

Syntax

```
if condition:
```

```
    indentedStatementBlockForTrueCondition
```

```
else:
```

```
    indentedStatementBlockForFalseCondition
```

These statement blocks
can have any number and
type of statements.

IF-ELSE Statement



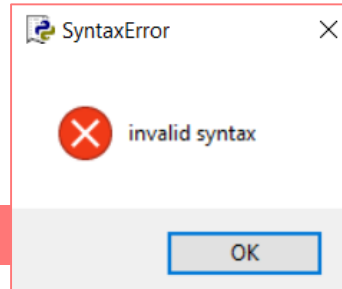
Examples

```
if a > b:
    print("a > b")
else:
    print("a <= b")
```



In Python, a colon marks the start of a block or suite.

```
a = (int)(input("please enter an integer: "))
b = (int)(input("please enter another integer: "))
if a > b:
    print ("a > b")
    print ("a > b")
else:
    print ("a <= b")
```



Message

Do not forget the colon.
Otherwise, it causes a
syntax error!

IF-ELSE Statement (Cont'd)



Example

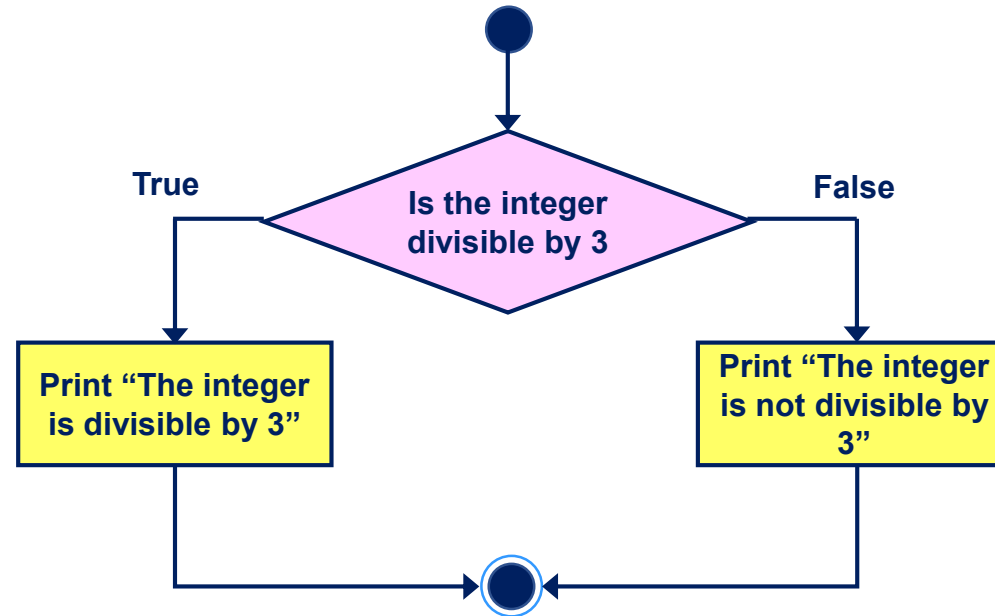
```
import math
radiusString = input("Enter the radius of your circle:")
radiusFloat = float(radiusString)
if radiusFloat > 0:
    circumference = 2 * math.pi * radiusFloat
    area = math.pi * radiusFloat * radiusFloat

    print("The circumference of your circle is: ",
          circumference, \
          ", and the area is:", area)
else:
    print("radius must be positive")
    print("please try again")
```



Application of IF-ELSE Statement

Determine if an Integer is Divisible by 3



Quick Try: Intelligent Guess



How to determine whether m is divisible by n ?

How to determine whether the remainder of m/n is zero in Python?

Operator	Meaning	Syntax	Example (x = 8, y = 3)
%	Modulus operator – returns the remainder of the division of left operand by the right	$x \% y$ (remainder of x/y)	$8 \% 3 = 2$

```
m = 8
n = 3
if m % n == 0:
    print(n, "is a divisor of", m)
else:
    print(n, "is not a divisor of", m)
```





Today is Tuesday. After 53 days, what day will it be?

Quick Check: Answer



Today is Tuesday. After 53 days, what day will it be?

Answer

Saturday

```
>>> 2 + 53 % 7
6
```

$$\begin{array}{r} 7 \overline{) 53} \\ \underline{49} \\ 4 \end{array}$$

Arithmetic Operators in Python

Operator	Meaning	Operator	Meaning
+	- Add two operands - Unary plus	%	Modulus - remainder of the division of left operand by the right
—	- Subtract the right operand from the left - Unary minus	//	Floor division (integer division) - the resultant value is a whole integer, although the result's type is not necessarily int
*	Multiply two operands	**	Exponent - the left operand is raised to the power of the right operand
/	Floating point division: divide the left operand by the right one (always results in a float)		

Arithmetic Operators in Python (Cont'd)

Operator	Meaning	Example	Output (x = 8, y = 3)
+	- Add two operands - Unary plus	$x + y$ $+x$	11 8
-	- Subtract the right operand from the left - Unary minus	$x - y$ $-x$	5 -8
*	Multiply two operands	$x * y$	24

Arithmetic Operators in Python (Cont'd)

Operator	Meaning	Example	Output (x = 8, y = 3)
/	Floating point division: divide the left operand by the right one (always results in a float)	x / y	2.6666666666666665
%	Modulus - remainder of the division of left operand by the right	x % y (remainder of x/y)	2
//	Floor division (integer division) - the resultant value is a whole integer, although the result's type is not necessarily int	x // y	2
**	Exponent - the left operand is raised to the power of the right operand	x ** y (x to the power of y)	512

Note: Division in Python

Differences between Python Versions **In Python 3.x: Integer/integer → Float**

- $8/4 \rightarrow 2.0$
- $8/3 \rightarrow 2.6666666666666665$

A trick in Python 3:

```
>>> 8.0/4.0
2.0
>>> 8/4
2.0
>>> int(8/4)
2
>>> int(8.0/4.0)
2
```

```
>>> 8//4
2
>>> 8/3
2.6666666666666665
>>> 8//3
2
```

Floor division

Note: Division by Zero

- $1 / 0$  Runtime error
- How about $1 \% 0$? Try it yourself.

```
>>> 1 / 0
Traceback (most recent call last):
  File "<pyshell#12>", line 1, in <module>
    1 / 0
ZeroDivisionError: division by zero
>>> 1 % 0
Traceback (most recent call last):
  File "<pyshell#13>", line 1, in <module>
    1 % 0
ZeroDivisionError: integer division or modulo by zero
```



Grace has \$25. She wants to buy movie tickets at \$3 each. Which of the following statements calculates the maximum number of tickets that she can purchase?

A. $25 / 3$

B. $25 \% 3$

C. $25 // 3$

D. $25 \%\% 3$

Quick Check: Answer



Grace has \$25. She wants to buy movie tickets at \$3 each. Which of the following statements calculates the maximum number of tickets that she can purchase?

A. $25 / 3$

B. $25 \% 3$

remainder

C. $25 // 3$

quotient

D. $25 \% 3$

Answer

C. $25 // 3$

Mixed Operations

- We have seen **8 / 3** and **8.0 / 4.0**.
- How about **8 / 3.0**? Here, there are **different data types**, i.e., **int and float**.
 - This is called a mixed operation.
 - Actually, there is no mixed operation.
 - Data are **implicitly** converted.

Python will **automatically convert** the data to the **most detailed** result.

- Thus, $8 \rightarrow 8.0$ and the result is 2.6666666.
- Detail: $\text{int} < \text{float}$.



Note: It is more detailed but (maybe) less precise.

Order of Calculation

General mathematical rules apply:

Operators	Description
()	Parentheses/ round brackets (grouping)
**	Exponentiation
+x, -x	Positive, negative
*, /, %	Multiplication, division, remainder
+, -	Addition, subtraction

Increase in priority ↑



Note: Always use parentheses if in doubt.

2 * 2 ** 2 vs. (2 * 2) ** 2

Augmented Assignments

- These operations are **shortcut**.
- They make the code easier to read.

Shortcut		Equivalence	
myInt	+= 2	↔	myInt = myInt + 2
myInt	-= 2	↔	myInt = myInt - 2
myInt	/= 2	↔	myInt = myInt / 2
myInt	*= 2	↔	myInt = myInt * 2
myInt	%= 2	↔	myInt = myInt % 2



What is the output of the following code?

```
x = 1.5  
y = 2  
x *= y  
print (x)
```



- A. 1.5 B. 2 C. 3.0 D. 3

Quick Check: Answer



What is the output of the following code?

```
x = 1.5  
y = 2  
x *= y  
print (x)
```



- A. 1.5 B. 2 C. 3.0 D. 3

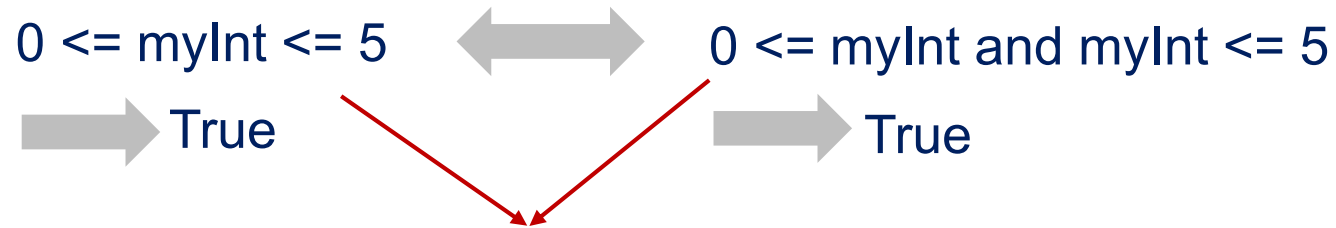
Answer

C. 3.0

```
x ← 1.5  
y ← 2  
x *= y ↔ x = x * y  
x = 1.5 * 2.0  
x = 3.0
```

Chained Comparisons

- In **Python** (but not in most languages), chained comparisons work just like you would expect in a mathematical expression.
- Say, **myInt** has a value of **5**:



Both have the same meaning, i.e., implicit “**and**”.



More Example

$0 \leq \text{myInt} \leq 5 > 10$

$\longrightarrow$ **False**

Just apply each operator to compare its two **neighbouring** values and then “and” the results.



```
grade = 87
if (80 < grade <= 90):
    print('very good')
if (90 < grade <= 100):
    print('excellent')
```

Precedence of Relational and Arithmetic Operators

Increase in priority ↑	Operators	Description
	()	Parentheses/ round brackets (grouping)
	**	Exponentiation
	+x, -x	Positive, negative
	*, /, %	Multiplication, division, remainder
	+, -	Addition, subtraction
	<, <=, >, >=, !=, ==	Comparisons
	not x	Boolean NOT
	and	Boolean AND
	or	Boolean OR

IF-ELIF-ELSE Statement - General Python Syntax

Syntax

```
if expression1:
```

```
    suite1
```

```
elif expression2:
```

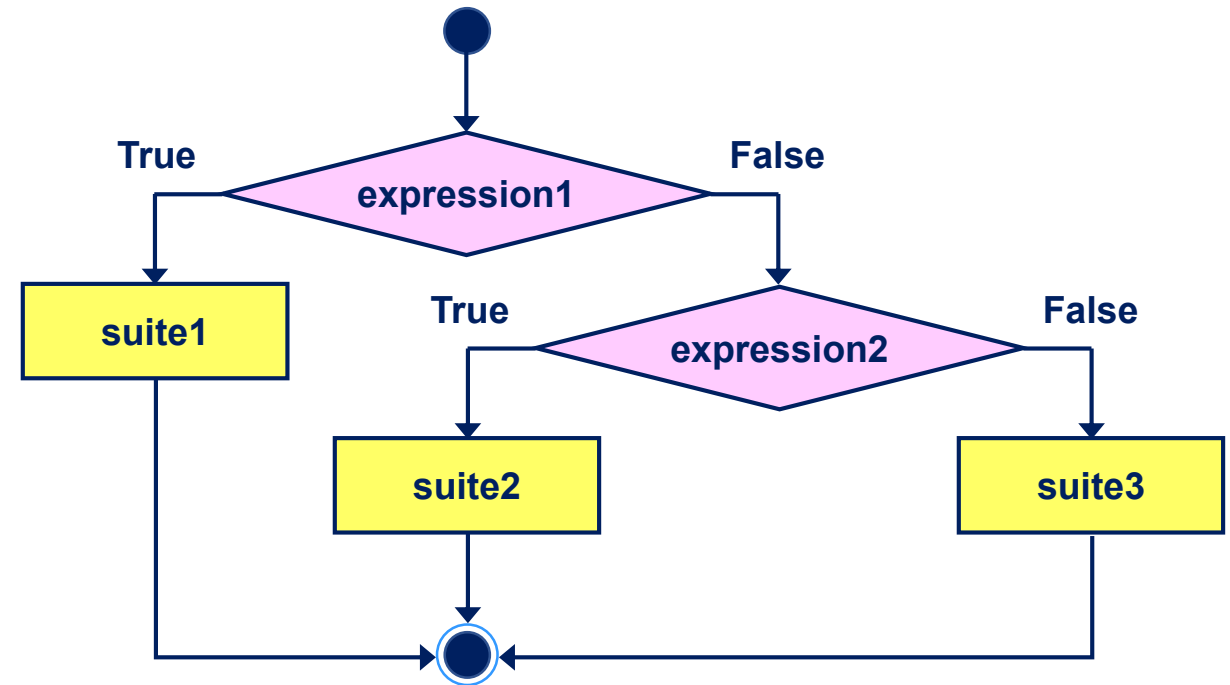
```
    suite2
```

```
else:
```

```
    suite3
```

Inside the False block,
you may have another
IF statement(s).

elif == else if



IF-ELIF-ELSE Statement (Cont'd)



Example

```
if a > b:
    print("a > b")
    print("case 1 here")
elif a < b:
    print("a < b")
    print("case 2 here")
else:
    print("a == b")
    print("case 3 here")
```



In Python, indentation defines a block. If you continue to use the same amount of indentation, it is still in the same block.

IF-ELIF-ELSE Statement (Cont'd)

Pseudocode

Whole statement {

```
IF condition1 is True THEN
    DO SUITE A
ELIF condition2 is True THEN
    DO SUITE B
ELIF condition3 is True THEN
    DO SUITE C
ELSE
    DO SUITE D # conditions 1,2,3 are all false
END IF
.....
```

} You may have many, many ELIF blocks.

IF-ELIF-ELSE Statement (Cont'd)



Further Example

```
mark = float(input('what is your mark?'))
```

```
if 90 <= mark <= 100:
```

```
    print('Congratulations. You received an A.')
```

```
elif 80 <= mark < 90:
```

```
    print('You received a B.')
```

```
elif 70 <= mark < 80:
```

```
    print('You received a C.')
```

```
elif 60 <= mark < 70:
```

```
    print('You received a D.')
```

```
else:
```

```
    print('oops, not good. Wish you can be better next time.')
```

Output:

What is your mark? 95

Congratulations. You received an A.

Output:

What is your mark? 55

oops, not good. Wish you can be better next time.



Is the following Python code correct?

```
mark = float(input('what is your mark?'))

if 90 <= mark <= 100:
    print('Congratulations. You received an A.')
if 80 <= mark < 90:
    print('You received a B.')
if 70 <= mark < 80:
    print('You received a C.')
if 60 <= mark < 70:
    print('You received a D.')
else:
    print('oops, not good. Wish you can be better next time.')
```



Quick Check: Answer



Is the following Python code correct?

```
mark = float(input('what is your mark?'))

if 90 <= mark <= 100:
    print('Congratulations. You received an A.')
if 80 <= mark < 90:
    print('You received a B.')
if 70 <= mark < 80:
    print('You received a C.')
if 60 <= mark < 70:
    print('You received a D.')
else:
    print('oops, not good. Wish you can be better next time.')
```



Output:
What is your mark? 95
Congratulations. You received an A.
oops, not good. Wish you can be
better next time.

Answer  **No**

```
if 90 <= mark <= 100:
    print('Congratulations. You
received an A.')
elif 80 <= mark < 90:
    print('You received a B.')
elif 70 <= mark < 80:
    print('You received a C.')
elif 60 <= mark < 70:
    print('You received a D.')
else:
    print('oops, not good. Wish you can
be better next time.')
```

```
if 90 <= mark <= 100:
    print('Congratulations. You
received an A.')
if 80 <= mark < 90:
    print('You received a B.')
if 70 <= mark < 80:
    print('You received a C.')
if 60 <= mark < 70:
    print('You received a D.')
else:
    print('oops, not good. Wish you can
be better next time.')
```

Nested IF Statement

Syntax

```
if condition1:
```

```
    if condition2:
```

```
        SUITE A
```

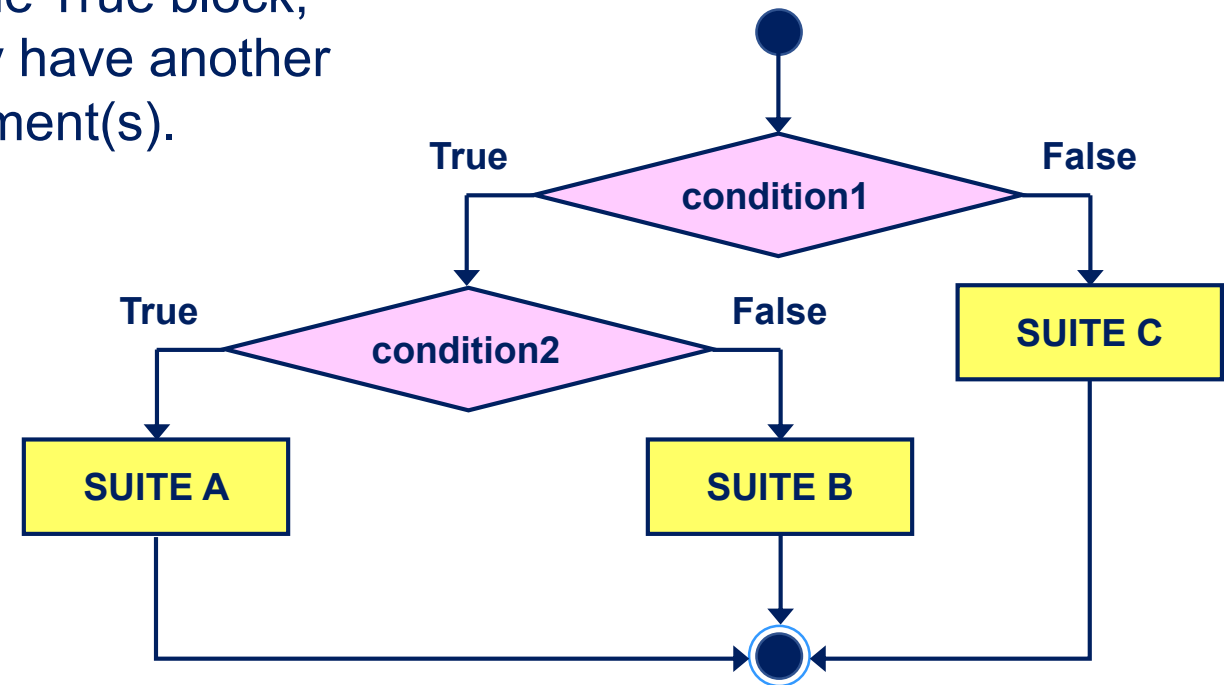
```
    else:
```

```
        SUITE B
```

```
else:
```

```
    SUITE C
```

Inside the True block,
you may have another
IF statement(s).



Nested IF Statement (Python)



Example

```
if a >= b:
    if a >= c:
        print("maximum value is ", a)
    else:
        print("maximum value is ", c)
else:
    if b >= c:
        print("maximum value is ", b)
    else:
        print("maximum value is ", c)
```



Proper indentation is
needed for the second
level.



Note: There is no new syntax here.

Nested IF Statement (Cont'd)



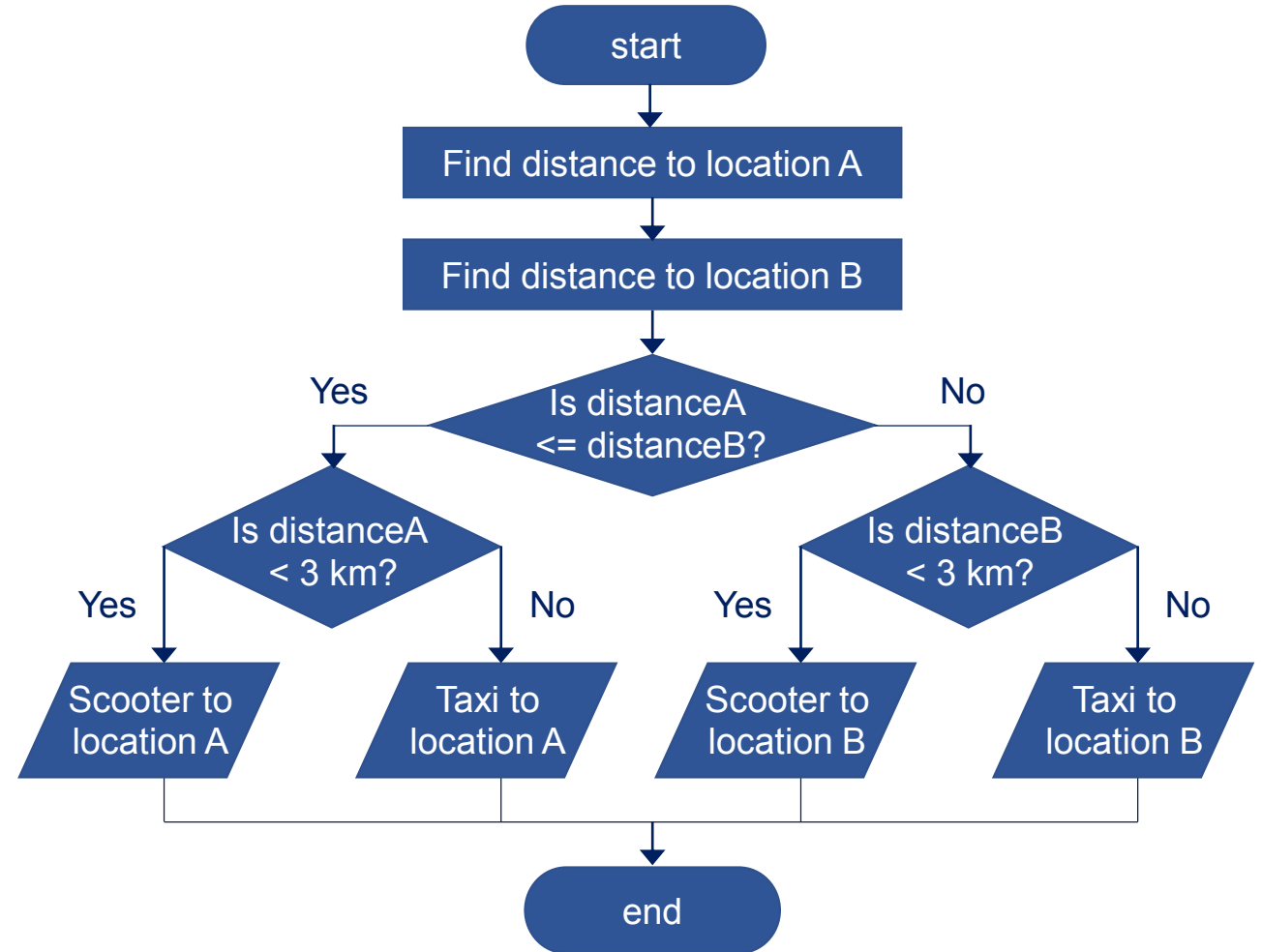
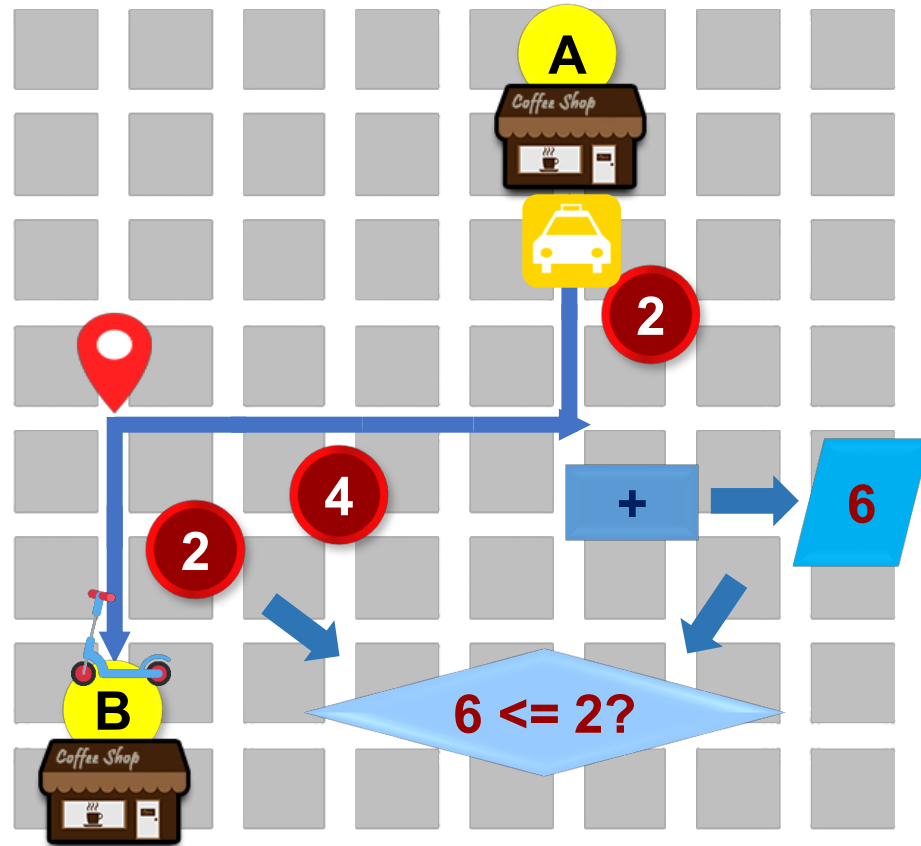
Further Example

```
mark = float(input('what is your mark?'))

if mark >= 60:
    if mark >= 70:
        if mark >= 80:
            if mark >= 90:
                print('Congratulations. You received an A.')
            else:
                print('You received a B.')
        else:
            print('You received a C.')
    else:
        print('You received a D.')
else:
    print('oops, not good. Wish you can be better next time.')
```



Scenario 6: Decide Transportation to the Nearer Coffee Shop



Scenario 6: Python Code

```
#My first Python application
#Date: 11/04/2018
#if-else; if-elif-else
horizonDistA = (int)(input("Read horizonDistA in meters"))
vertDistA = (int)(input("Read vertDistA in meters"))
distA = horizonDistA + vertDistA
print("dist from home to A is ", distA, "m")
horizonDistB = (int)(input("Read horizonDistB in meters"))
vertDistB = (int)(input("Read vertDistB in meters"))
distB = horizonDistB + vertDistB
print("dist from home to B is ", distB, "m")

if distA <= distB:
    if distA < 3000:
        print("You are suggested to ride a e-scooter to location A")
    else:
        print("You are suggested to take a taxi to location A")
elif distB < 3000:
    print("You are suggested to ride a e-scooter to location B")
else:
    print("You are suggested to take a taxi to location B")
print("Thank you for using this application")
```



IF-ELSE

```
if condition:  
    indentedStatementBlockForTrueCondition  
else:  
    indentedStatementBlockForFalseCondition
```

(No *NEW* syntax)

Nested IF

```
if condition1:  
    if condition2:  
        SUITE A  
    else:  
        SUITE B  
else:  
    SUITE C
```

IF-ELIF-ELSE

```
if expression1:  
    suite1  
elif expression2:  
    suite2  
else:  
    suite3
```

**More on Selection
(Branching) SYNTAX**

Relational and Arithmetic Operations

Increase in priority

Operators	Description
()	Parentheses/ round brackets (grouping)
**	Exponentiation
+x, -x	Positive, negative
*, /, %	Multiplication, division, remainder
+, -	Addition, subtraction
<, <=, >, >=, !=, ==	Comparisons
not x	Boolean NOT
and	Boolean AND
or	Boolean OR

References for Images

No.	Slide No.	Image	Reference
1	All Examples		Survey icon [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/survey-icon-survey-icon-2316468/ .
2	All Python codes		Python Logo [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/language-logo-python-2024210/ .
3	21, 36		By User:Bobarino - Made by following Information.png, CC BY-SA 3.0, retrieved April 18, 2018 from https://en.wikipedia.org/w/index.php?curid=9180601 .
4	6, 39		Search [Online Image]. Retrieved April 18, 2018 from https://pixabay.com/en/database-search-database-search-icon-2797375/ .